

UNIVERSIDADE POSITIVO

ALEXANDRE HENNING WAHL
FELIPE PORTO CALDEIRA DO NASCIMENTO
GABRIELL DE SOUZA ZAPPELINI
VITOR ALEXANDRE HASSELMANN DE OLIVEIRA
WILLIAM FELIPE COSCODAI

**INTERPOLAÇÃO DE FRAMES POR ATENÇÃO ESPACIAL E CANAIS DE
ATENÇÃO**

CURITIBA - PR

2025

ALEXANDRE HENNING WAHL
FELIPE PORTO CALDEIRA DO NASCIMENTO
GABRIELL DE SOUZA ZAPPELINI
VITOR ALEXANDRE HASSELMANN DE OLIVEIRA
WILLIAM FELIPE COSCODAI

**INTERPOLAÇÃO DE FRAMES POR ATENÇÃO ESPACIAL E CANAIS DE
ATENÇÃO**

Documento apresentado como requisito parcial de aprovação da disciplina de Trabalho de Conclusão de Curso II do Bacharelado em Ciência da Computação e do Bacharelado em Engenharia da Computação da Universidade Positivo.

Orientador (a): Prof Msc Leandro Escobar

CURITIBA - PR

2025

RESUMO

A interpolação de *frames* é um processo utilizado no processamento de vídeos, amplamente utilizado para gerar quadros intermediários que aumentam a taxa de quadros e melhoram a qualidade visual em diversas aplicações. Os métodos baseados em fluxo óptico, embora eficazes em muitas situações, apresentam limitações notáveis, principalmente em cenários com movimentos rápidos, oclusões ou variações de iluminação, além de demandar um alto custo computacional. Nesse contexto, este trabalho tem como objetivo propor um modelo de interpolação de quadros baseado em mecanismos de atenção espacial e de atenção de canal, removendo sua dependência no uso de fluxo óptico. A abordagem técnica se fundamenta em arquiteturas de *deep learning*, onde redes neurais convolucionais são empregadas para extrair e combinar recursos relevantes dos quadros de entrada. Os mecanismos de atenção espacial ressaltam regiões importantes na imagem, enquanto a atenção de canais pondera a relevância de diferentes mapas de características, permitindo a construção precisa do quadro intermediário. Para melhorar a qualidade das imagens geradas, o modelo incorpora uma função de perda denominada perda perceptual na rede VGG19, de modo a preservar detalhes visuais e texturas realistas. Os resultados esperados incluem interpolação de vídeo mais robusta, com preservação de detalhes e baixa presença de artefatos. As aplicações potenciais dessa abordagem abrangem entretenimento audiovisual, realidade virtual e reanimação de vídeos antigos, dentre outros. Em suma, o método proposto oferece uma alternativa promissora conciliando qualidade visual com eficiência computacional.

Palavras-chave: interpolação de *frames*; atenção espacial; atenção de canais; redes neurais convolucionais; aprendizado profundo; perda perceptual

ABSTRACT

Frame interpolation is a process used in video processing, widely used to generate intermediate frames that increase the frame rate and improves visual quality in several applications. Optical flow-based methods, although effective in many situations, have notable limitations, especially in scenarios with fast movements, occlusions or lighting variations, in addition to demanding a high computational cost. In this context, this work proposes a frame interpolation model based on spatial attention and channel attention mechanisms, removing its dependence on the use of optical flow. The technical approach will be based on deep learning architectures, where convolutional neural networks are employed to extract and combine relevant features from the input frames. Spatial attention mechanisms highlight important regions in the image, while channel attention weighs the relevance of different feature maps, allowing an accurate reconstruction of the intermediate frame. To improve the quality of the generated images, the model incorporates a loss function called perceptual loss in the VGG19 network, in order to preserve visual details and realistic textures. The expected results include more robust video interpolation, with detail preservation and artifact reduction. Potential applications of this approach include audiovisual entertainment, virtual reality and reanimation of old videos, among others. In short, the proposed method offers a promising alternative combining visual quality with computational efficiency.

Keywords: frame interpolation; spatial attention; channel attention; convolutional neural networks; deep learning; perceptual loss

LISTA DE FIGURAS

Figura 1 - Arquitetura de Redes Neurais Artificiais.....	14
Figura 2 - Arquitetura VGG-19 de ZHOU.....	19
Figura 3 - Loss Network.....	20
Figura 4 - Feature maps extraídos da VGG19 de KIM e LEE.....	21
Figura 5 - Arquitetura de Redes Neurais Convolucionais.....	23
Figura 6 - Ilustração do Módulo de Atenção Espacial.....	24
Figura 7 - Ilustração do Módulo de Atenção de Canal.....	26
Figura 8 - Arquitetura do modelo SIMON.....	32
Figura 9 - Arquitetura do modelo HUAIZU.....	33
Figura 10 - Arquitetura modelo de WENBO.....	34
Figura 11 - Arquitetura do modelo HYEONGMIN.....	35
Figura 12 - Arquitetura do modelo CHOI (2020).....	36
Figura 13 - Arquitetura do modelo XIANHANG e ZHENZHONG.....	37
Figura 14 - Estrutura Analítica do Escopo.....	40
Figura 15 - Diagrama de Processo (Treinamento).....	46
Figura 16 - Diagrama de Processo(Interpolação).....	47
Figura 17 - Janela de configurações de baixa fidelidade.....	48
Figura 18 - Janela de apuração de baixa fidelidade.....	49
Figura 19 - Arquitetura do modelo inicial.....	51
Figura 20 - InvNorm.....	51
Figura 21 - Resultado da Interpolação.....	53
Figura 22 - Close-up dos resultados.....	54
Figura 23 - Arquitetura do modelo final.....	55
Figura 24 - UpConvNorm.....	56
Figura 25 - Comparativo entre modelos.....	57
Figura 26 - Close-up dos modelos.....	58
Figura 27 - Interface de configuração.....	59
Figura 28 - Seção do modelo.....	60
Figura 29 - Seção do vídeo de entrada.....	60
Figura 30 - Seção do diretório de saída.....	61
Figura 31 - Aba de Apuração.....	61

SUMÁRIO

1. INTRODUÇÃO.....	7
1.1. Necessidade.....	7
1.2. Problema.....	7
1.3. Justificativa.....	8
1.4. Objetivo Geral.....	10
1.4.1. Objetivos Específicos.....	10
2. REFERENCIAL TEÓRICO.....	11
2.1. Pixel.....	11
2.2. Kernel.....	11
2.3. Patch.....	12
2.4. Stride.....	12
2.5. Decibel (dB).....	12
2.6. Peak Signal-to-Noise Ratio.....	12
2.7. Fluxo Óptico.....	13
2.8. Redes Neurais Artificiais.....	14
2.9. Funções de Perda.....	15
2.9.1. Structural Similarity Index (SSIM).....	15
2.9.2. Charbonnier Loss.....	17
2.9.3. Perceptual Loss com VGG19.....	18
2.10. Redes Neurais Convolucionais.....	23
2.11. Atenção Espacial em Visão Computacional.....	24
2.12. Atenção de Canal em Visão Computacional.....	25
2.13. Convolução Adaptativa.....	27
2.14. Vetores de Desvio.....	27
2.15. Mapas de Profundidade.....	28
2.16. Conexão de Atalho (skip connection).....	28
2.17. Grupos Residuais.....	28
2.18. Blocos Residuais.....	29
2.19. Pixel Shuffle.....	29
2.20. Acurácia em Interpolação de Frames.....	30
3. REVISÃO DA LITERATURA.....	31
3.1. Metodologia de Pesquisa.....	31
3.2. Trabalhos Correlatos.....	32
3.3. Síntese da Literatura Correlata.....	37
4. ESCOPO.....	39
4.1.1. Interface.....	40
4.1.2. Seleção de Modelo.....	40
4.1.3. Treinamento de Modelo Pelo Usuário.....	40
4.1.4. Carregar Vídeos.....	41

4.1.5. Geração de Quadros.....	41
4.1.6. Taxa de Interpolação Personalizável.....	41
4.1.7. Comparação Entre Vídeos.....	41
4.1.8. Exportação de Vídeo.....	41
4.1.9. Câmera Lenta.....	41
5. PLANEJAMENTO.....	42
5.1. Tecnologias.....	42
5.1.1. Dados de Treinamento.....	42
5.1.2. Tecnologias de Desenvolvimento.....	42
5.1.3. Utilização de Rede Convolucionais.....	43
5.1.4. Funções de Perda Utilizadas.....	43
5.1.5. Atenção Espacial.....	44
5.1.6. Atenção de Canal.....	44
5.1.7. Inclusão de Conexões de Atalho - skip connection.....	44
5.1.8. Inclusão de Blocos Residuais.....	44
5.1.9. Inclusão de Grupos Residuais.....	44
5.1.10. Inclusão de Pixel Shuffle.....	45
5.2. Descrição dos Procedimentos do Sistema.....	45
5.3. Protótipo de Telas.....	47
5.3.1. Configurações.....	47
5.3.2. Apuração.....	48
6. RESULTADOS.....	50
6.1. Resultados do Escopo e Planejamento da Solução.....	50
6.2. Arquitetura Inicial.....	50
6.3. Apuração do Modelo Inicial.....	53
6.4. Arquitetura Final.....	54
6.5. Apuração do Modelo Final.....	56
6.6. Interface de Alta Fidelidade.....	59
7. CONCLUSÃO.....	63
7.1. Desafios no Desenvolvimento.....	64
7.2. Trabalhos Futuros.....	66
7.3. Considerações Finais.....	66
REFERÊNCIAS (alfabética).....	68
REFERÊNCIAS (cronológica).....	73

1. INTRODUÇÃO

O crescimento de vídeos trouxe consigo uma exigência cada vez maior de qualidade visual. Entre os aspectos mais valorizados está a fluidez de movimentos na tela, fator essencial para uma experiência de visualização mais natural. Com este contexto, será aprofundado o assunto para uma síntese do que é necessário para uma possível contribuição à esta forma de mídia.

1.1. Necessidade

Diante do cenário tecnológico atual, o consumo e a produção de conteúdo audiovisual crescem exponencialmente. Estima-se, por exemplo, que vídeos online representem mais de 82% de todo o tráfego de internet dos consumidores, esse panorama reflete um público que não apenas consome mais vídeos, mas que exige vídeos com níveis de qualidade elevados para retenção de engajamento (DEGUZMAN, 2024).

Em telas contemporâneas de alta definição e alta taxa de atualização, por exemplo, telas *Full Hd* e acima de 60Hz de frequência, movimentos suaves tornaram-se essenciais para conteúdo de qualidade, pois vídeos em baixas taxas de quadros aparentam travar ou “pularem” aos olhos do espectador. Em outras palavras, um vídeo com FPS (*frames per second*, quadros por segundo) baixo não possui fluidez (MACKIN; ZHANG; BULL, 2015).

Essa necessidade de aprimorar a fluidez do vídeo abrange desde o entretenimento doméstico, como jogos, cinema e realidade virtual, em que conteúdos em alta resolução e alta taxa de quadros oferecem experiências mais envolventes, até aplicações profissionais, nas quais a alta taxa de quadros aprimora a clareza e o conforto visual.

1.2. Problema

Apesar da demanda por vídeos mais fluidos, muitos conteúdos ainda sofrem com descontinuidades visuais devido às limitações na captação original. Equipamentos antigos, câmeras de baixo custo ou mesmo cenários adversos de

gravação frequentemente produzem vídeos em baixa taxa de quadros ou com irregularidades entre os quadros.

Dessa forma, movimentos que deveriam ser contínuos passam a ocorrer de forma fragmentada. Dependendo do caso, gravações legadas ou de baixo *frame rate* tornam os movimentos aos saltos e pouco naturais na tela. Além disso, condições de filmagem desafiadoras, como pouca iluminação ou objetos em deslocamento rápido, podem introduzir *motion blur* (borrão de movimento) e até perda de *frames*, degradando ainda mais a continuidade visual.

A solução clássica para restaurar a continuidade perdida em vídeos é a interpolação de quadros baseada em fluxo óptico (*optical flow*), isto é, em estimativa de campo de movimento entre *frames*. Algoritmos consagrados de fluxo óptico, como o de Farnebäck et al (2003), procuram mapear o deslocamento de pixels de um quadro para o próximo, de modo a gerar quadros intermediários seguindo esse movimento estimado.

Entretanto, a abordagem baseada em fluxo óptico enfrenta diversos desafios técnicos. Primeiramente, obter um fluxo óptico suficientemente preciso em todas as regiões da imagem é computacionalmente custoso, muitas vezes exigindo algoritmos de otimização global complexos (BRIEDIS, 2021). Em seguida, esses métodos podem falhar em casos de movimentos muito rápidos, oclusões ou mudanças bruscas nas cenas, situações em que o fluxo óptico encontra dificuldades para estimar o movimento das imagens, gerando borrões, distorções e “fantasmas”.

Embora a interpolação via fluxo óptico seja a abordagem predominante e já tenha alcançado resultados notáveis, ela permanece limitada por alta complexidade computacional, dependência de modelos de IA (inteligência artificial) de grande porte para atingir níveis altos de acurácia, e sensibilidade a condições adversas de captura (BRIEDIS, 2021).

1.3. Justificativa

Diante das carências explicitadas no uso do fluxo óptico, justifica-se a criação de processos de interpolação de *frames* que dispensem o uso deste,

buscando manter a acurácia e qualidade na geração de *frames* intermediários. A premissa é que, ao remover a necessidade de calcular um campo de fluxo a cada par de *frames*, pode-se reduzir significativamente a complexidade algorítmica e a demanda computacional do processo. No entanto, métodos alternativos de interpolação de *frames* ainda enfrentam limitações, como a geração de artefatos visuais em quadros sintéticos, incluindo padrões ou distorções que não refletem com precisão a realidade da cena, o que pode comprometer a fidelidade visual e a naturalidade do vídeo gerado.

Briedis (2021) afirma que existem métodos baseados em convoluções adaptativas ou transformações no domínio de frequência têm demonstrado desempenho próximo dos métodos com fluxo óptico, porém com menor custo computacional.

Considerando as limitações dos métodos tradicionais baseados em fluxo óptico, como a alta complexidade algorítmica, o elevado custo computacional e a sensibilidade a falhas, torna-se pertinente buscar abordagens alternativas que sejam mais eficientes, independentes e robustas. Busca-se, por meio deste modelo, alcançar uma maior precisão na geração de *frames*, promovendo melhorias significativas na preservação de detalhes e na fidelidade dos *frames* gerados.

Diante desse cenário, propõe-se o desenvolvimento de um sistema de interpolação de *frames* que dispense o uso de fluxo óptico, com o objetivo de tornar a solução mais acessível a diferentes contextos de aplicação. Espera-se, com isso, contribuir para a criação de métodos mais robustos, adaptáveis e eficientes, que atendam às demandas atuais de produção e processamento de vídeo, inclusive em ambientes com recursos limitados ou variações nas condições de captação.

1.4. Objetivo Geral

O projeto tem como objetivo, implementar um modelo de interpolação de *frames* com mecanismos de atenção para melhorar a qualidade da geração de *frames* intermediários em vídeos, por meio de métricas objetivas e perceptuais comumente utilizadas, visando medir a eficácia na reconstrução visual.

1.4.1. Objetivos Específicos

- Usar *Deep Learning* como base arquitetural para a geração de *frames* intermediários em vídeos, utilizando sua capacidade de extração de características espaciais para prever *frames* com maior coerência visual e temporal;
- Aplicar canais de atenção ao modelo convolucional, com o objetivo de reforçar a importância de canais específicos na extração de características relevantes, otimizando o processo de geração de *frames* intermediários mais consistentes;
- Aplicar o atenção espacial à arquitetura convolucional para realçar regiões espaciais significativas nas imagens de entrada, permitindo ao modelo focar em áreas críticas na reconstrução dos *frames* intermediários;
- Adaptar e treinar o modelo de geração de *frames* utilizando um conjunto de dados apropriado;
- Usar o *Perceptual Loss* com VGG19 para comparar características de alto nível extraídas de uma rede neural pré-treinada, o qual foi treinada para perceber padrões visuais similares aos percebidos por humanos;
- Avaliar o desempenho do modelo de geração de *frames* utilizando métricas objetivas.

2. REFERENCIAL TEÓRICO

Nesta seção são apresentados os conceitos e as definições dos elementos adotados no desenvolvimento da pesquisa, iniciando pelos termos mais elementares, passando pelos algoritmos e modelos quantitativos e finalizando com as técnicas mais elaboradas de treinamento de redes neurais.

2.1. Pixel

Pixel (*picture element*) é a menor unidade discreta que compõe uma imagem digital, normalmente armazenando valores de cor RGB (vermelho, verde e azul) ou intensidade de brilho. Cada pixel corresponde a um ponto da imagem, e o valor atribuído a ele representa a cor ou intensidade naquele ponto específico (GEEKSFORGEEKS, 2024). Para imagens em tons de cinza, há apenas um valor de intensidade por pixel e para imagens coloridas, cada pixel possui três valores RGB. A grade de pixels forma a malha da imagem completa.

2.2. Kernel

Kernel ou filtro refere-se a uma matriz de pesos usada em operações de convolução sobre imagens. Também chamado de máscara, um *kernel* tem dimensões menores que a imagem e é aplicado deslocando-o através dos pixels da imagem. Na operação de convolução, o *kernel* atua como um detector de padrões: a cada posição, seus valores são multiplicados pelos pixels correspondentes da imagem local (*patch*) e somados, produzindo um valor de saída. Esse valor compõe o mapa de características filtrado. Em CNNs (Redes Neurais Convolucionais), os *kernels* são os parâmetros aprendidos da rede: cada *kernel* aprende a realçar um tipo de característica (por exemplo, borda, textura, etc.) (LECUN et al, 1998). Uma característica importante é que o mesmo *kernel* é aplicado em toda a imagem, o que reduz a quantidade de pesos necessários (LECUN et al, 1998).

2.3. Patch

Em processamento de imagens, um *patch* representa uma pequena sub-região contígua de uma imagem. Um exemplo seria o *patch* 5x5, um bloco de 5 pixels de altura e largura extraído de uma imagem maior. Nas convoluções, cada aplicação do *kernel* corresponde ao processamento de um *patch* do mesmo tamanho do filtro. A essência é que um patch encapsula informações locais da imagem, como textura ou cor de uma área específica. Em síntese, *patch* é qualquer bloco reduzido extraído de uma imagem, tratado como unidade autônoma para análise ou processamento (MILVUS, 2025).

2.4. Stride

O termo *stride* refere-se ao número de pixels pelos quais o filtro convolucional se desloca sobre a imagem de entrada durante a operação de convolução. Um *stride* igual a 1 significa que o filtro move um pixel por vez, resultando em uma maior sobreposição entre regiões vizinhas. Já valores maiores de *stride* causam uma redução mais agressiva na dimensão espacial da imagem de saída, funcionando também como uma forma de subamostragem. A escolha do *stride* afeta de maneira inversamente proporcional a resolução espacial das ativações e o custo computacional da rede (MILVUS, 2025).

2.5. Decibel (dB)

O decibel (dB) é uma unidade logarítmica utilizada para expressar razões de potência ou intensidade. O resultado em dB indica a semelhança visual em que valores maiores de PSNR (*Peak Signal-to-Noise Ratio*) representam maior fidelidade, por exemplo, 40 dB corresponde a uma qualidade melhor do que 30 dB (FIVEABLE, 2025).

2.6. Peak Signal-to-Noise Ratio

PSNR (*Peak Signal-to-Noise Ratio*) (1) é uma métrica quantitativa usada para avaliar a qualidade de reconstrução ou compressão de imagens e vídeos, comparando um resultado gerado com uma imagem de referência considerada de

qualidade máxima. PSNR significa Razão Pico de Sinal-Para-Ruído, e é definida matematicamente a partir da razão entre a potência máxima do sinal e a potência do ruído presente. Em contexto de imagens digitais, o sinal seria a imagem de referência e o ruído seria o erro da imagem gerada em relação à referência (CLOUARD, 2023; FIVEABLE, 2025). A fórmula PSNR pode ser assim representada:

$$PSNR(x, y) = 10 \log_{10} \left(\frac{L^2}{MSE(x, y)} \right) \quad (1)$$

onde:

- x, y são as imagens de entrada (predita e referência), com mesmos tamanhos e intervalo de valores de pixel;
- O fator 10 escala o logaritmo para a conversão de decibéis.
- $\log_{10}$ é o logaritmo de base 10, convertendo a razão de potências em decibéis (dB);
- L é o valor máximo possível de um pixel (por exemplo, 255 para imagens de 8 bits por canal);
- $MSE(x, y)$ é o erro quadrático médio entre x e y ;

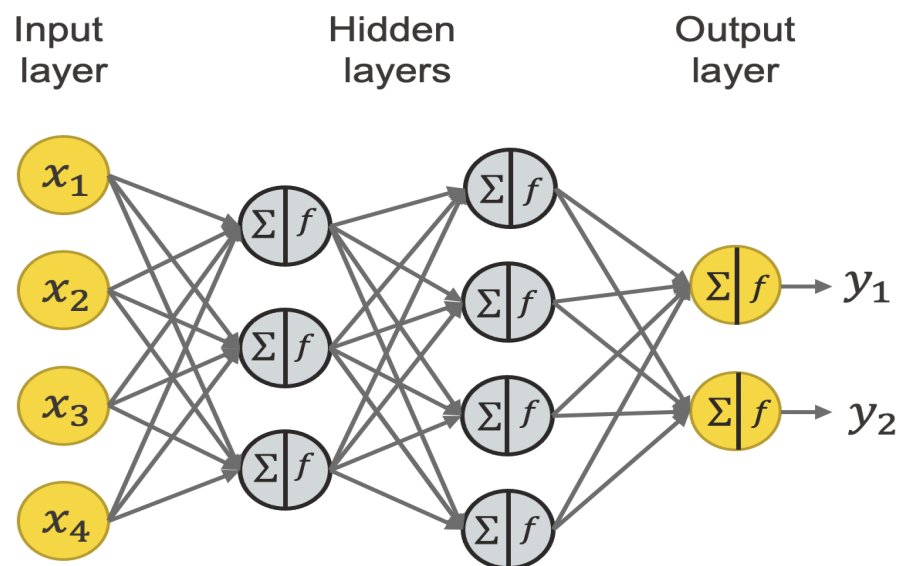
2.7. Fluxo Óptico

Fluxo óptico (*optical flow*) é o campo de vetores que representa o deslocamento perceptível de pixels entre dois *frames* consecutivos de um vídeo. Para cada pixel em um *frame*, o fluxo óptico estima um vetor indicando para onde aquele pixel se moveu no próximo *frame*. Esse movimento pode ser causado pelo deslocamento de objetos na cena ou pelo movimento da própria câmera (FARNEBÄCK, 2003). O fluxo óptico é amplamente usado para estimar movimento em visão computacional, embora exija resolver problemas de ambiguidade (por exemplo, menor número de equações que variáveis) e lidar com oclusões.

2.8. Redes Neurais Artificiais

Redes Neurais Artificiais (RNAs) são modelos computacionais inspirados na estrutura e funcionamento das redes de neurônios biológicos do cérebro (HAYKIN, 2009) (Figura 1).

Figura 1 - Arquitetura de Redes Neurais Artificiais



Fonte: HENRYRU (2022)

Em uma RNA, diversos neurônios artificiais são interconectados em camadas. Cada neurônio recebe sinais de neurônios da camada anterior, realiza uma soma ponderada desses sinais e aplica uma função de ativação, transmitindo o resultado para neurônios da próxima camada. Por meio do ajuste dos pesos das conexões durante um processo de treinamento supervisionado, a rede aprende a associar entradas a saídas desejadas, minimizando um valor de erro definido por uma função de perda (HAYKIN, 2009; GOODFELLOW, 2016).

2.9. Funções de Perda

Em aprendizado supervisionado, a função de perda quantifica o erro entre a saída predita de um modelo e o valor esperado (*ground truth*), orientando o processo de otimização para minimizar esse erro (GOODFELLOW, 2016). Em visão computacional, empregam-se funções de perda especializadas para priorizar a qualidade visual das imagens (JOHNSON, 2016; WANG, 2004). Essas funções podem incorporar perceptibilidade ou similaridade estrutural, indo além da simples discrepância de intensidade. Uma função de perda utilizada em tarefas de regressão e reconstrução de imagens é o Erro Quadrático Médio (*Mean Squared Error*, MSE) (2), definida como:

$$MSE(x, y) = \frac{1}{N} \sum_{i=1}^N (x_i - y_i)^2 \quad (2)$$

onde:

- x, y representam os valores de entrada e predito pelo modelo, respectivamente;
- x_i e y_i são os valores dos pixels originais e preditos pelo modelo, respectivamente. Por exemplo, x_1 representa o valor do primeiro pixel de entrada e y_1 representa o valor predito correspondente;
- N representa o número total de elementos.

2.9.1. Structural Similarity Index (SSIM)

A métrica SSIM foi proposta para avaliar qualidade de imagens de forma mais alinhada à percepção humana. Em vez de medir apenas a diferença absoluta por pixel, o SSIM foca em semelhanças estruturais entre duas imagens, decompondo a comparação em três componentes: luminosidade, contraste e estrutura. A luminosidade refere-se à comparação do brilho médio das duas imagens, enquanto o contraste refere-se à comparação da variação das intensidades e a estrutura avalia a correlação pixel a pixel entre as imagens. Esses três componentes são combinados para resultar no índice SSIM geral, que varia de -1 a 1, onde valores mais próximos de 1 indicam maior similaridade estrutural (WANG, 2004). A fórmula de SSIM(3) pode ser expressada como:

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (3)$$

onde:

- x, y representam os valores de entrada e predito pelo modelo, respectivamente;
- μ_x, μ_y são intensidades médias (médias locais) dos pixels em uma janela pequena ao redor de cada ponto das imagens de x e y ;
- C_1, C_2 são constantes de estabilização. Essas constantes evitam divisões por zero quando σ_x^2, σ_y^2 ou μ_x^2, μ_y^2 ficam muito pequenos.
- σ_{xy} é a covariância local. Indica o grau de correlação entre variações das janelas de x e y . Se, onde a janela de x sobe, y também sobe, a covariância é positiva e alta;

- σ_x^2, σ_y^2 são as variâncias locais das janelas de x e y . A variância mede o quanto os valores de pixel dentro daquela mesma janela variam em torno da média. Se todos os pixels tiverem valores muito parecidos, a variância será baixa (baixa textura/baixo contraste);

2.9.2. Charbonnier Loss

Segundo BARRON (2017), *Charbonnier Loss* (4) é uma função de perda utilizada em visão computacional, principalmente em problemas de estimação de movimento e reconstrução de imagens, por sua natureza robusta e estável. É essencialmente uma forma suave da perda L1 (*L1 Loss*). Diferentemente do erro L2 (quadrático), que penaliza fortemente grandes erros, e do erro L1 puro, que é linear mas não diferenciável em 0, a perda de *Charbonnier* comporta-se como L2 para erros muito pequenos e aproxima L1 para erros grandes. Em outras palavras, para discrepâncias pequenas entre predição e verdade, ela é suave e evita oscilações nas atualizações de peso, enquanto para discrepâncias grandes ela cresce aproximadamente linearmente, reduzindo a influência de *outliers* extremos no gradiente. Essa combinação de suavização e robustez traz estabilidade numérica ao treinamento e torna o modelo menos sensível a ruídos ou erros esporádicos nos dados. Na prática, a perda de *Charbonnier* tem sido utilizada na estimação de fluxo óptico, disparidade e em redes de super-resolução e interpolação de *frames*, onde se deseja um equilíbrio entre minimizar erros médios e preservar detalhes sem ser demasiado influenciado por discrepâncias isoladas. A fórmula *Charbonnier Loss* pode ser expressa por:

$$Charbonnier(x, y) = \frac{1}{N} \sum_{i=1}^N \sqrt{(x_i - y_i)^2 + \epsilon^2} \quad (4)$$

onde:

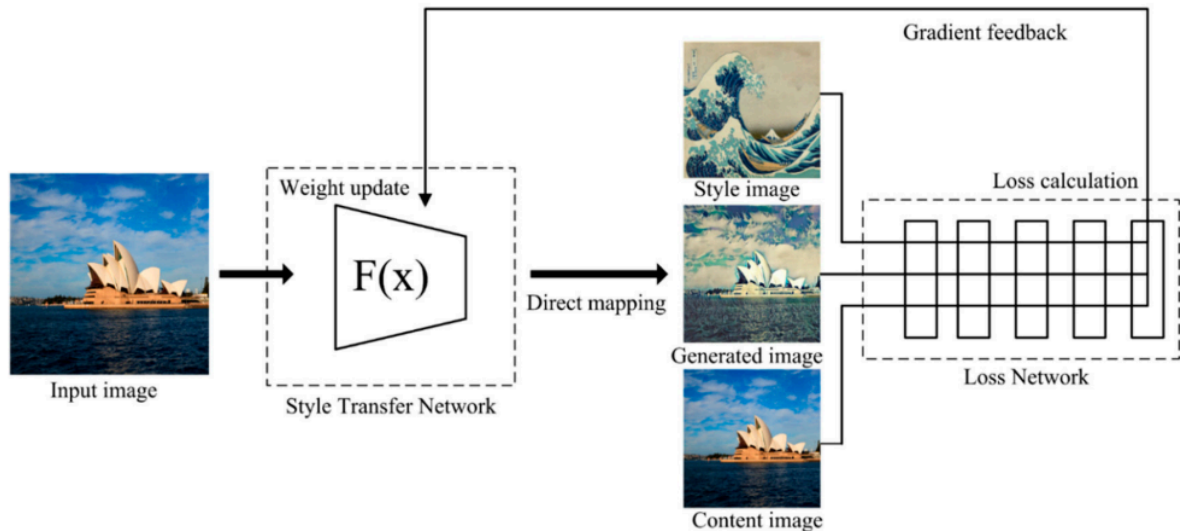
- x e y são tensores (ou vetores) de valores previstos por um modelo e de referência, respectivamente;
- x_i e y_i são os componentes individuais de x e y , para $i=1,\dots,N$. Por exemplo, x_1 e y_1 seriam os valores predito e verdadeiro do primeiro pixel;
- N é o número total de elementos em x e y (por exemplo, número de pixels ou características);
- ε é uma constante de regularização, que evita que o termo dentro da raiz seja zero, garantindo diferenciabilidade;
- $\sqrt{(x_i - y_i)^2 + \varepsilon^2}$ é a penalização robusta para o erro $x_i - y_i$. Quando a diferença é menor que ε , o termo dentro da raiz é dominado por ε^2 , fazendo com que a penalização se comporte de forma quadrática suave.

2.9.3. *Perceptual Loss* com VGG19

Visto no trabalho de Johnson et al (2016), *Perceptual Loss* (perda perceptual) é uma classe de função de perda que busca medir a diferença entre duas imagens de uma maneira mais correlacionada à percepção visual humana, usando redes neurais pré-treinadas para comparar características de alto nível em vez de pixels individuais.

Conforme ilustra a Figura 2, a *Perceptual Loss* com VGG19 usa como ponto de partida a *Style Transfer Network*, que recebe a imagem de entrada, *Input image*, e gera a imagem processada, *Generated image*.

Figura 2 - Arquitetura VGG-19 de ZHOU

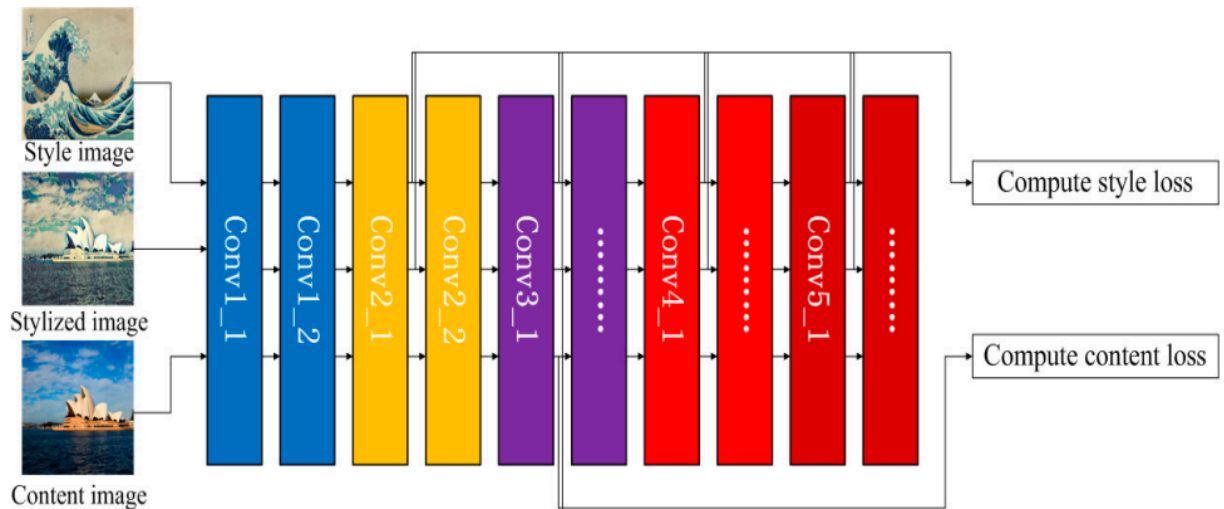


Fonte: ZHOU et al (2019)

Em seguida, essa saída é comparada em duas frentes de interesse no *Loss Network* vista na Figura 3, cujas camadas intermediárias funcionam como alvos:

- *Content image* fornece o conteúdo desejado. São extraídos seus mapas de ativação em uma camada profunda *conv3_1* e é calculada a diferença (MSE) destes mapas com os da imagem processada, gerando a perda de conteúdo (*Compute content loss*).
- *Style image* fornece o estilo a ser imitado. Em várias camadas rasas e médias (*conv2_1*, *conv3_1*, *conv4_1*, *conv5_1*), são extraídas matrizes de estilo tanto da imagem processada quanto da imagem estilizada e é calculada a diferença entre elas, resultando nas perdas de estilo (*Compute style loss*)

Figura 3 - Loss Network



Fonte: Adaptado de ZHOU et al (2019)

A soma ponderada dessas perdas orienta o treinamento da *Style Transform Network*, de modo que a imagem processada preserve o conteúdo da imagem de conteúdo e adote o estilo da imagem de estilo conforme medido pelos recursos extraídos pela VGG-19, resultando a imagem estilizada (*Stylized image*).

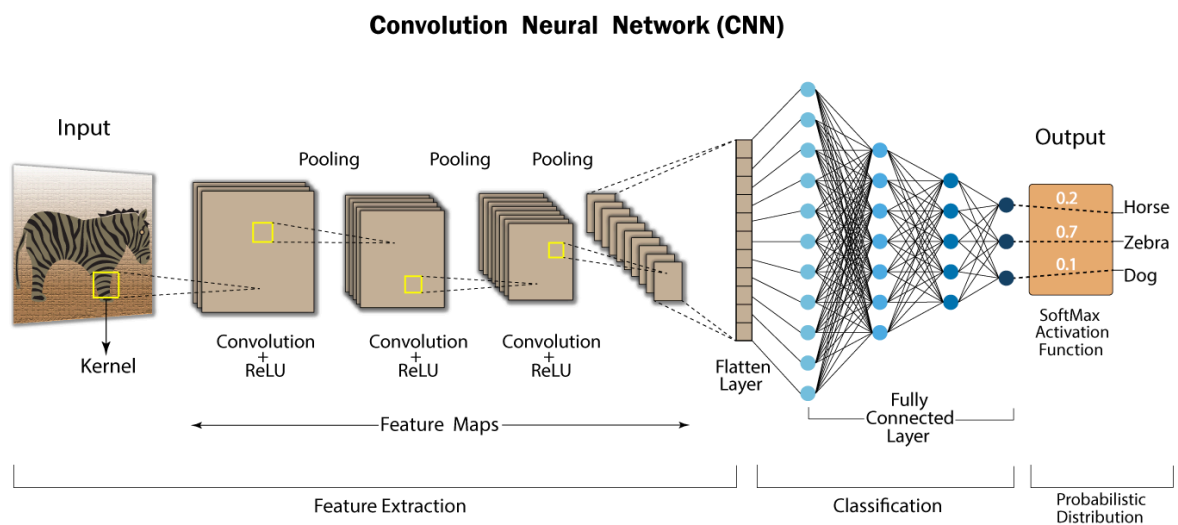
Segundo Ledig et al (2017), a ideia da VGG19 surgiu inicialmente em tarefas como transferência de estilo e super-resolução de imagens, onde se observou que otimizar apenas erros pixel-a-pixel (L1, L2) frequentemente resultava em imagens borradas ou com menos detalhes, apesar de terem PSNR alto. Isso ocorre porque uma pequena mudança de posição de um detalhe (por exemplo, um deslocamento de 1 pixel em uma textura fina) pode causar um grande erro MSE, mesmo que visualmente a qualidade permaneça boa. A *Perceptual Loss* contorna esse problema ao comparar diferenças perceptuais e semânticas: em vez de punir diretamente diferenças em intensidade de cada pixel, ela faz com que a rede aprenda a produzir imagens cuja representação interna em um modelo como VGG19 seja similar à da imagem alvo. As camadas da VGG19 atuam como detectores de características de baixo, médio e alto nível. Por exemplo, camadas iniciais capturam bordas e texturas locais, enquanto camadas mais profundas capturam aspectos globais e semânticos como formas de objetos e rostos.

- x, y são as imagens de entrada (predita e referência), cada uma com valores originalmente em $[-1, 1]$;
- $x' = \text{MeanShift}(\text{clamp}(\frac{x+1}{2}, 0, 1))$ (e similar para y') aplica a conversão $(x + 1)/2$ para mapear valores em $[0, 1]$, o clamp para eliminar excessos e manter os valores em $[0, 1]$, e por fim a subtração média $\mu = (0.485, 0.456, 0.406)$ e divisão pelo desvio padrão $\sigma = (0.229, 0.224, 0.225)$ do *ImageNet*, em que são utilizados para homogeneizar a entrada, centralizar os valores em torno de zero e evitar pixels com valores muito altos que podem saturar as funções de ativação de algumas camadas;
- $\phi_l(\cdot)$ representa a passagem da imagem x' ou y' pela VGG19 cortada até a camada l resultando em um *feature map*;
- l define até onde a rede é cortada para extrair o mapa:
 - $l = '22' \rightarrow \text{conv2_2}$,
 - $l = '33' \rightarrow \text{conv3_3}$,
 - $l = '44' \rightarrow \text{conv4_4}$,
 - $l = '54' \rightarrow \text{conv5_4}$.
- $C_l H_l W_l$ são, respectivamente, o número de canais, altura e largura do mapa de características $\phi_l(\cdot)$;
- A soma tripla e divisão por $C_l H_l W_l$ implementam a média do erro quadrático (MSE) entre os mapas de características de x e y .

2.10. Redes Neurais Convolucionais

Conforme ZHAO (2024) e SPARELLO (2022), as Redes Neurais Convolucionais são uma classe especial de RNA projetada para lidar eficientemente com dados que tenham estrutura espacial local, como imagens (Figura 5).

Figura 5 - Arquitetura de Redes Neurais Convolucionais



Fonte: Sparello (2022)

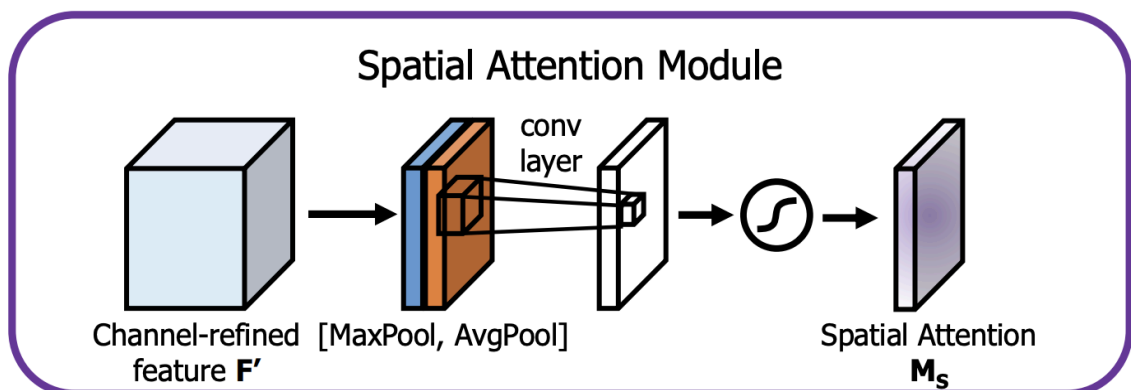
ZHAO (2024) informa que antes das CNNs, redes totalmente conectadas sofriam ao processar imagens pois atribuíam um peso independente para cada pixel de entrada, resultando em um número massivo de parâmetros e ignorando a proximidade espacial entre pixels. As CNNs resolveram esse problema introduzindo dois conceitos-chave: conectividade local e compartilhamento de parâmetros. Em vez de cada neurônio conectar-se a todos os pixels de entrada, cada neurônio convolucional conecta-se apenas a uma pequena região local da imagem, explorando a localidade espacial e reconhecendo padrões visuais locais. Além disso, os mesmos filtros são aplicados em todas as posições da entrada por meio da operação de convolução, movendo o filtro sobre a imagem inteira. Esse compartilhamento de pesos garante que o filtro reconheça o mesmo tipo de característica visual independentemente da posição.

O autor também afirma que uma CNN típica é composta por camadas convolucionais seguidas opcionalmente de camadas de *pooling*. As camadas convolucionais aplicam múltiplos filtros aprendidos que extraem características, produzindo ativações altas onde esses padrões são encontrados. As camadas de *pooling* reduzem a resolução espacial dos mapas de características, resumindo informações e introduzindo invariância a pequenas variações de posição. Em comparação às redes tradicionais, as CNNs generalizam melhor em problemas de visão computacional graças a essas propriedades, além de utilizarem muito menos pesos devido ao compartilhamento.

2.11. Atenção Espacial em Visão Computacional

Atenção espacial é um mecanismo que permite à rede neural aprender onde focar sua capacidade de processamento em uma imagem ou mapa de características (WOO, 2018)(Figura 6).

Figura 6 - Ilustração do Módulo de Atenção Espacial



Fonte: WOO (2018)

Conforme mostra a Figura 6, o módulo de atenção espacial inicia-se com o *channel-refined feature F'* , que é o mapa de características de entrada já ponderado pelos pesos de atenção de canal. Em seguida, aplicam-se duas operações de redução espacial: o *MaxPool*, que retém em cada posição o valor máximo de ativação para capturar as regiões de maior resposta, e o *AvgPool*, que calcula a

média das ativações para fornecer uma visão mais estável do contexto. Esses dois mapas são então concatenados e alimentados em uma *conv layer* de dimensão 7x7 (ou similar), cuja saída passa por uma função sigmoide para gerar o mapa de atenção espacial M_s que realça selectivamente as regiões mais relevantes antes de multiplicá-lo de volta em F' .

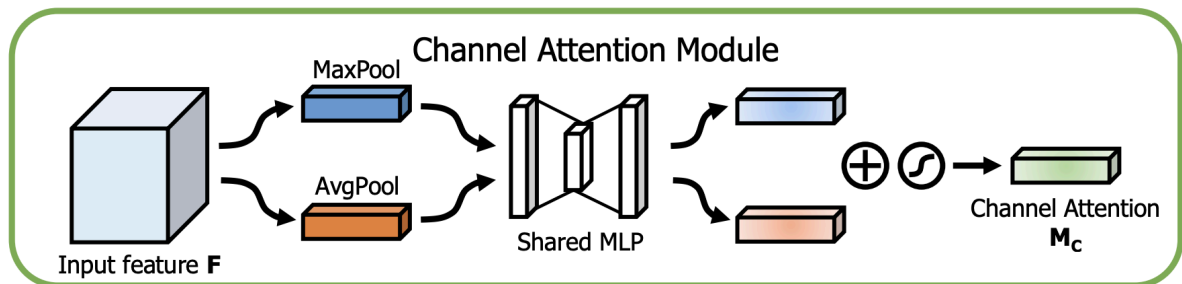
A ideia deriva do conceito geral de atenção em redes neurais, que prioriza certos elementos dos dados de entrada. No caso espacial, o objetivo é atribuir pesos diferentes a distintas regiões da imagem, destacando as regiões mais relevantes para a tarefa em questão e suprimindo as menos informativas (WOO, 2018). Em problemas de visão computacional, nem todos os pixels ou regiões da imagem contribuem igualmente; por exemplo, ao interpolar *frames* de vídeo, regiões em torno de contornos de objetos em movimento podem exigir mais atenção do que fundos estáticos.

Um módulo típico de atenção espacial recebe um mapa de características e produz um mapa de atenção bidimensional. Para calcular esse mapa, abordagens comuns incluem agrupar informações ao longo dos canais para determinar a importância de cada localização. Uma técnica consagrada é combinar um mapa de média e um mapa de máximo calculados pixel-a-pixel ao longo de todos os canais, e então usar uma pequena camada convolucional para gerar o mapa de atenção final a partir dessa combinação (WOO, 2018). Esse mapa resultante, com valores entre 0 e 1 obtidos via uma função sigmoide, indica onde enfatizar ou atenuar. Posições com valores próximos de 1 serão realçadas (multiplicadas por pesos altos), enquanto posições com valores próximos de 0 serão suprimidas (WOO, 2018).

2.12. Atenção de Canal em Visão Computacional

Enquanto a atenção espacial atua sobre posições na imagem (largura e altura), a atenção de canal atua sobre os canais dos mapas de características, isto é, foca quais filtros ou canais são mais informativos para a tarefa corrente. Em uma CNN, cada canal de um *feature map* pode ser visto detectando um tipo específico de padrão ou atributo (HU et al, 2018; WOO, 2018) (Figura 7).

Figura 7 - Ilustração do Módulo de Atenção de Canal



Fonte: WOO (2018)

Como ilustrado na Figura 7, o módulo de atenção de canal começa com o *input feature F*, que é o mapa de características original da camada convolucional. Em paralelo, aplica-se um *MaxPool*, que extrai o valor máximo de ativação em cada canal, e um *AvgPool*, que calcula a média das ativações, produzindo dois vetores de dimensão $C \times 1 \times 1$ que resumem globalmente cada canal. Esses vetores passam por um *Shared MLP* (rede neural de duas camadas com pesos compartilhados), que reduz e depois expande sua dimensionalidade, gerando dois novos vetores de canal reponderados. Os vetores são então combinados via soma e normalizados com uma função sigmoide, produzindo o mapa de atenção de canal M_c , que é multiplicado elemento a elemento em F para reforçar os canais mais informativos.

A atenção de canal permite à rede realçar seletivamente os canais mais relevantes e suprimir os menos úteis dada uma determinada entrada (HU et al, 2018). O funcionamento de um módulo de atenção de canal inicia-se com um *squeeze* espacial: realiza-se uma agregação (por exemplo, pooling global) em cada canal, condensando a informação de cada mapa completo em um único valor representativo por canal. O resultado são vetores de um valor por canal que capturam, em média, o quanto aquele filtro ativou e potencialmente a força máxima daquela ativação (HU et al, 2018). Em seguida, esses vetores passam por uma pequena rede (geralmente algumas camadas densas) que produz um vetor de pesos de atenção de tamanho único para todos os canais. Esses pesos, normalmente normalizados entre 0 e 1 (via sigmoide), formam o mapa de atenção de canal.

Por fim, aplica-se esse mapa multiplicando cada canal original pelo peso correspondente (HU et al, 2018). Intuitivamente, se a imagem de entrada contém certos padrões que um subconjunto de filtros da CNN reconhece bem, a atenção de canal atribui pesos maiores a esses filtros, reforçando suas respostas. Canais menos relevantes para aquela imagem receberão pesos menores, sendo atenuados (HU et al, 2018).

2.13. Convolução Adaptativa

Convolução adaptativa refere-se a métodos em que os filtros e suas posições de amostragem são gerados dinamicamente conforme o conteúdo local, em vez de usar um *kernel* fixo. Nesses métodos, a rede aprende a ajustar pesos e *offsets* (vetores de desvio) de cada filtro para capturar movimentos complexos sem calcular o fluxo óptico explícito. Abordagens como as convoluções adaptativas de SIMON et al (2017) e as *Deformable Convolutional Networks* de DAI et al (2017) introduzem deslocamentos aprendidos nos filtros. O AdaCoF proposto por HYEONGMIN et al (2020), que generaliza esse conceito: para cada pixel intermediário, o modelo prevê tanto um *kernel* específico quanto múltiplos *offsets*, combinando informações dos quadros de entrada de maneira flexível (SIMON et al, 2017). Em suma, a convolução adaptativa permite que o núcleo de convolução ajuste dinamicamente suas posições de amostragem conforme o conteúdo, aumentando a capacidade de lidar com movimentos não lineares.

2.14. Vetores de Desvio

Vetores de desvio são parâmetros de deslocamento espacial utilizados no processo de interpolação de vídeo para indicar de quais posições dos *frames* de entrada devem ser extraídas informações ao compor cada pixel do *frame* intermediário. Em métodos avançados de interpolação, como o AdaCoF, esses vetores são aprendidos pela rede. Por exemplo, no AdaCoF, para cada pixel intermediário a rede estima múltiplos vetores de deslocamento juntamente com coeficientes de *kernel*. Esses vetores permitem mostrar valores em posições possivelmente fracionárias dos *frames* de origem e combinar múltiplas amostras vizinhas para preservar detalhes finos. Assim, comparado ao fluxo óptico tradicional,

o uso de múltiplos *offsets* por pixel aumenta a flexibilidade do modelo para lidar com movimentos complexos.(SIMON et al, 2017; CHOI et al, 2020).

2.15. Mapas de Profundidade

Mapas de profundidade são representações em imagem onde cada pixel codifica a distância do objeto naquele ponto até a câmera. Esses mapas podem ser obtidos diretamente por algoritmos de visão computacional e redes neurais a partir de imagens 2D. Como carregam explicitamente a geometria tridimensional da cena, mapas de profundidade fornecem atributos espaciais ricos, facilitando a reconstrução 3D e servindo como base para tarefas que exigem compreensão da estrutura e posição dos objetos (MOSTAFA, 2020). Nas CNNs de visão, o uso de mapas de profundidade confere um viés geométrico adicional à extração de características, permitindo capturar relações espaciais profundas além da análise puramente bidimensional.

2.16. Conexão de Atalho (*skip connection*)

Segundo HE et al (2016) *skip connection* é o termo usado para descrever uma ligação direta que pula uma ou mais camadas em uma rede neural, conectando a saída de uma camada anterior à entrada de uma camada mais à frente. Em redes convolucionais residuais, o *skip connection* geralmente assume a forma de uma soma direta da saída de um bloco de camadas com sua entrada original, implementando uma identidade que atravessa o bloco. O efeito dessa conexão direta é mitigar problemas de gradiente e permitir que informações não modificadas fluam através da rede mesmo conforme ela se aprofunda. Assim, camadas intermediárias aprendem apenas a diferença necessária (o residual), enquanto o *skip connection* garante que se essa diferença for zero a informação original seguirá intacta.

2.17. Grupos Residuais

Grupos residuais são estruturas formadas por múltiplos blocos residuais empilhados, complementados por uma conexão de atalho (*skip connection*) que envolve todo o grupo. Em outras palavras, dentro de um grupo residual cada bloco realiza sua transformação com um *skip connection* interno, e além disso o próprio

grupo possui um atalho ligando sua entrada diretamente à saída do último bloco. Essa organização em hierarquia residual em residual permite treinar redes neurais convolucionais ainda mais profundas de forma eficiente, pois facilita o fluxo de gradientes em diferentes escalas de profundidade (HE et al, 2016; ZHANG et al, 2018).

2.18. Blocos Residuais

HE et al (2016) também afirma que, blocos residuais são um tipo de módulo introduzido nas CNNs para enfrentar a degradação de desempenho em redes muito profundas. Um bloco residual típico consiste em uma sequência de camadas cuja saída é somada à entrada original do bloco por meio de uma conexão de atalho. Dessa forma, o bloco aprende a função residual, isto é, o ajuste necessário em relação à sua entrada, em vez de tentar aprender um mapeamento completamente novo. Essa simples alteração estrutural traz dois benefícios principais: evita a perda de sinal nas camadas profundas (o gradiente pode fluir diretamente pelo atalho) e permite que, caso as camadas internas não aprendam nada útil, o bloco residual ainda se comporte como uma identidade, passando a informação adiante sem prejudicá-la.

2.19. *Pixel Shuffle*

Os autores SHI, W et al (2016) afirma que *Pixel Shuffle* é uma operação utilizada em redes neurais convolucionais para aumentar a resolução espacial de mapas de características de forma eficiente, especialmente em tarefas de super-resolução de imagens. Diferentemente de técnicas tradicionais de *upsampling*, como convoluções transpostas, o *Pixel Shuffle* rearranja os pixels dos mapas de características de baixa resolução para construí-los em uma imagem de alta resolução. Por exemplo, ao ampliar uma imagem por fator 2, a camada *Pixel Shuffle* pegará grupos de 4 valores em canais distintos e os redistribui em um bloco 2x2 na saída, dobrando a altura e largura enquanto reduz o número de canais. Essa estratégia, também conhecida como *sub-pixel convolution*, permite que a maior parte do processamento seja feita ainda no espaço de baixa resolução, agregando eficiência computacional e evitando artefatos visuais comuns na deconvolução.

2.20. Acurácia em Interpolação de Frames

No contexto de redes neurais a acurácia é uma métrica utilizada para avaliar o quão próximo um modelo está de acertar suas previsões em relação ao resultado esperado. Em tarefas de classificação, por exemplo, a acurácia mede a porcentagem de exemplos previstos corretamente pelo modelo sobre o total de exemplos avaliados. Assim, quanto maior a acurácia, mais o modelo está respondendo conforme o esperado (GOODFELLOW, 2016).

A acurácia ganha nuances dependendo da tarefa, por exemplo, em visão computacional, ela pode se referir não apenas ao acerto ou erro como em tarefas de classificação, mas à proximidade ou coerência dos resultados. Esse também é o caso da acurácia na geração de conteúdos como vídeos e áudios, onde passa-se a levar em conta aspectos como a fidelidade, consistência visual, ausência de artefatos e preservação de características originais (JOHNSON et al., 2016).

Quando se trata de interpolação de *frames*, a tarefa do modelo não é rotular ou classificar, mas criar *frames* intermediários a partir de *frames* originais. A acurácia nesse contexto, passa a englobar critérios que vão além de números exatos, envolvendo a preservação da fluidez do movimento, integridade visual de objetos e a coerência cronológica. Dessa forma, a acurácia deixa de ser uma métrica exata e passa a ser medida por métricas que avaliam aproximações. Métricas como o SSIM (3) são utilizadas para esta finalidade, pois comparam a similaridade entre o *frame* previsto e o esperado (WANG et al., 2004). Um ótimo resultado é frequentemente refletido em valores de SSIM em torno de 0.95, que garantem que os resultados sejam visualmente mais coerentes com o frame original, tornando muito difícil de se perceber distorções na imagem através da visão humana (FLYNN, J.R. et al.).

3. REVISÃO DA LITERATURA

A revisão dos trabalhos correlatos se deu mediante uma pesquisa bibliográfica exploratória seguida da leitura e resumo dos materiais analisados.

3.1. Metodologia de Pesquisa

A pesquisa bibliográfica foi conduzida a partir de termos específicos como *frame interpolation*, *optical flow*, *adaptive convolution video interpolation*, *spatial attention convolutional networks*, *channel attention video frame* e *deep learning video enhancement*, aplicados de forma combinada. Cada consulta retornou diversos artigos, cujos títulos e resumos foram inicialmente examinados para identificar trabalhos que propuseram métodos de interpolação de *frames* sem depender de fluxo óptico e que apresentassem avaliações objetivas por métricas como PSNR e SSIM.

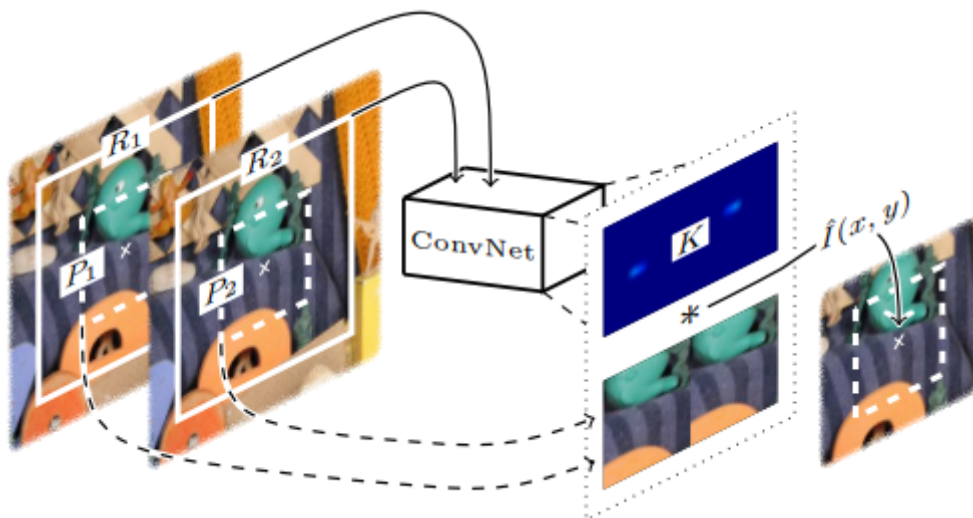
Foram incluídos estudos que utilizavam fluxo óptico quando havia alguma técnica complementar aplicada em conjunto ou quando o processo de execução e suas limitações eram descritos de forma clara, a fim de contextualizar os avanços sobre esse tipo de abordagem.

Para a etapa de inclusão foram selecionados artigos publicados entre 2017 e 2022, com descrição da arquitetura, dos procedimentos de treinamento e de teste, e com resultados numéricos reproduzíveis. Os artigos encontrados foram analisados para extrair informações sobre estrutura de rede, funções de perda utilizadas juntamente com motivos da utilização das mesmas, conjuntos de dados empregados e resultados de PSNR/SSIM.

3.2. Trabalhos Correlatos

SIMON et al (2017) propuseram uma rede neural para estimar um par de kernels de convolução posicionais para cada pixel do *frame* intermediário a ser gerado, onde kernels aplicados sobre patches dos frames anterior e posterior determinam as contribuições de cada pixel vizinho na síntese do novo quadro, realizando a interpolação via combinação ponderada de vizinhanças (Figura 8).

Figura 8 - Arquitetura do modelo SIMON

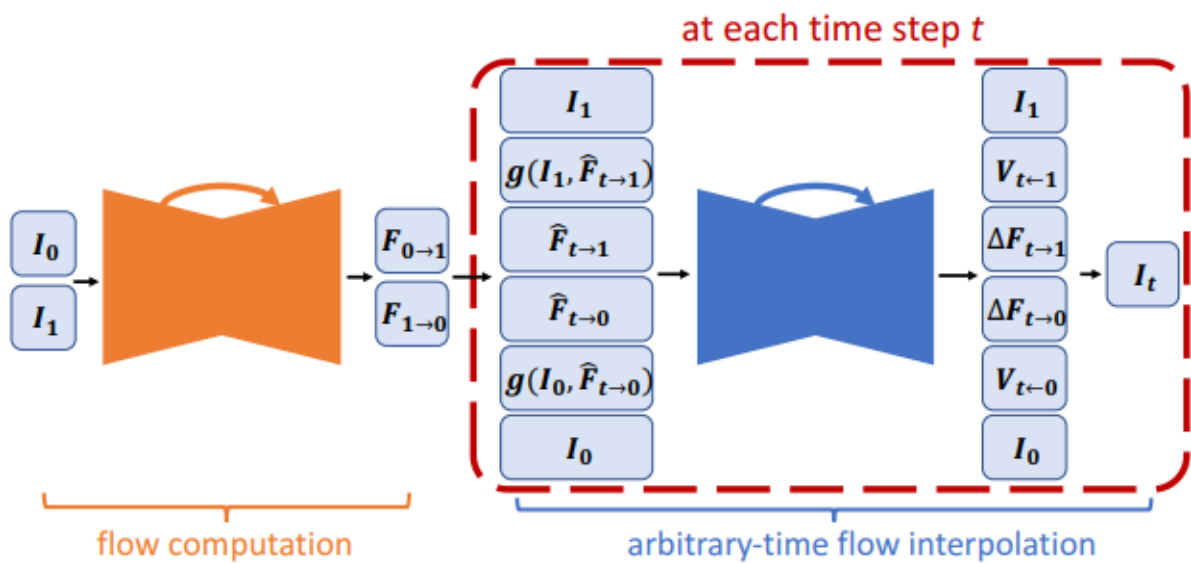


Fonte: SIMON et al (2017)

Contudo, o tamanho finito do kernel limita a amplitude de movimento que pode ser capturada. Por exemplo, deslocamentos maiores que 51 pixels, excedem a capacidade de amostragem do modelo e kernels maiores implicam aumento substancial no custo computacional e na memória exigida. Com essa abordagem, evitou-se a dependência de fluxo óptico explícito, inaugurando a linha de métodos de interpolação baseada em convolução adaptativa. Os autores demonstraram resultados de alta fidelidade para movimentos dentro do limite do *kernel*, alcançando métricas de qualidade competitivas (PSNR/SSIM) em comparação aos métodos baseados em fluxo óptico da época, apesar das restrições impostas pelo tamanho do kernel.

HUAIZU et al (2018) computa a bi-direção do fluxo óptico entre as imagens usando a arquitetura *U-Net*, com o intuito de utilizar uma CNN de ponta a ponta com comprimento variável a fim de ser uma aplicação de interpolação de *frames* para qualquer que seja o *frame-rate* do vídeo desejado para realizar um efeito de câmera lenta (Figura 9).

Figura 9 - Arquitetura do modelo HUAIZU

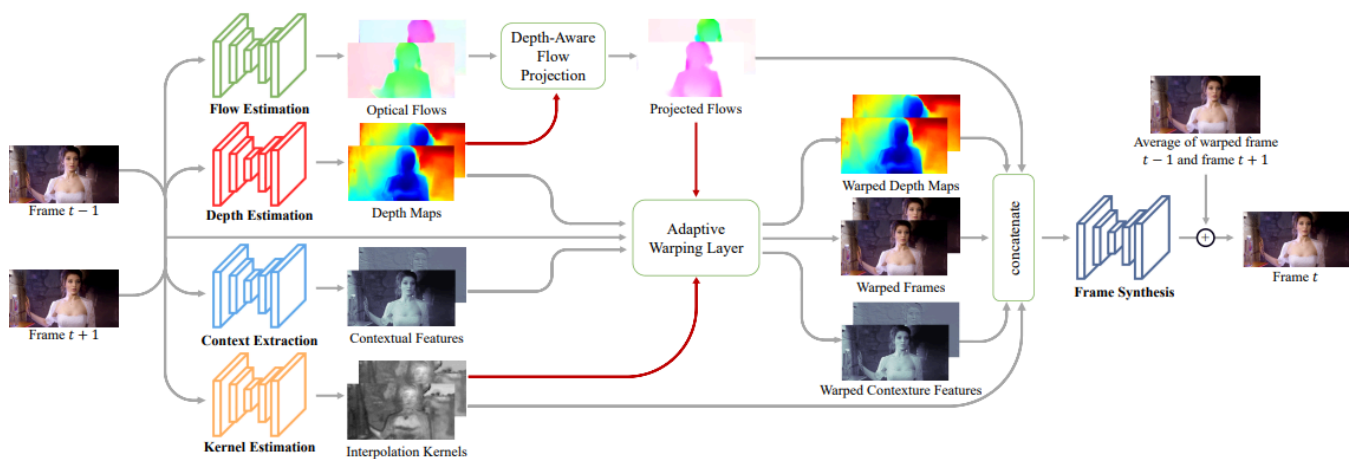


Fonte: HUAIZU et al (2018)

Por conta da alta taxa de *frames* a serem interpolados como a necessidade de refinar estes mesmos *frames* pela limitação de não compreender com exatidão movimentos bruscos nas imagens, é uma aplicação que acaba por gastar muito poder computacional. Nos experimentos, o método proposto alcançou *frames* interpolados de alta qualidade de forma consistente, superando abordagens de interpolação única e mantendo coerência espacial e temporal mesmo ao gerar múltiplos *frames* intermediários em sequência.

No trabalho de WENBO et al (2019), é visto a utilização deste se especializando na área de mapas de profundidade em conjunto com recursos contextuais baseado em fluxo óptico e interpolação local de *kernels* para sintetizar novos *frames*, com a idealização de gerar estes mesmos com um grande senso de movimento por conta do efeito de profundidade (Figura 10).

Figura 10 - Arquitetura modelo de WENBO



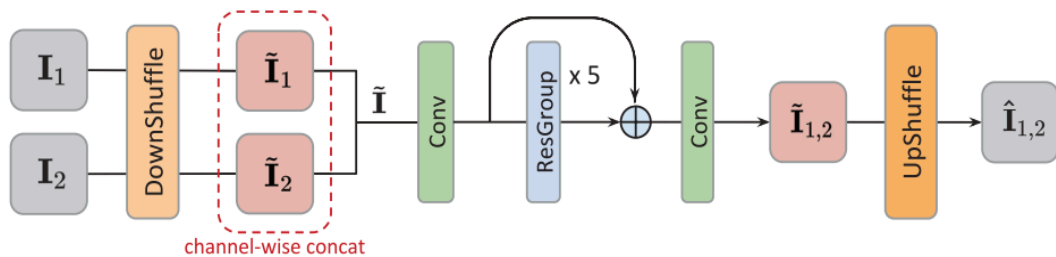
Fonte: WENBO et al (2019)

Em contrapartida, seu foco em propor um maior senso de profundidade funciona, em grande parte, em imagens com objetos perto da tela, resultando em mapas de profundidade mal estimados para casos à parte do proposto. Além disso, a arquitetura requer o cálculo de diversas entradas adicionais, como a estimativa de profundidade, extração de contexto, fluxo óptico e estimativa de *kernels*, que irão inevitavelmente aumentar a complexidade computacional do modelo.

Quantitativamente, o modelo proposto apresentou desempenho favorável frente aos métodos convencionais, demonstrando melhor preservação de detalhes e menos artefatos em diversos cenários, especialmente aqueles com oclusões, de acordo com os resultados reportados pelos autores.

No trabalho de CHOI et al (2020) foi proposto o CAIN (Channel Attention Is All You Need), um modelo de interpolação de quadros que dispensa o uso de fluxo óptico e introduz mecanismos de atenção de canal como principal meio de orientação da síntese do *frame* intermediário (Figura 12).

Figura 12 - Arquitetura do modelo CHOI (2020)

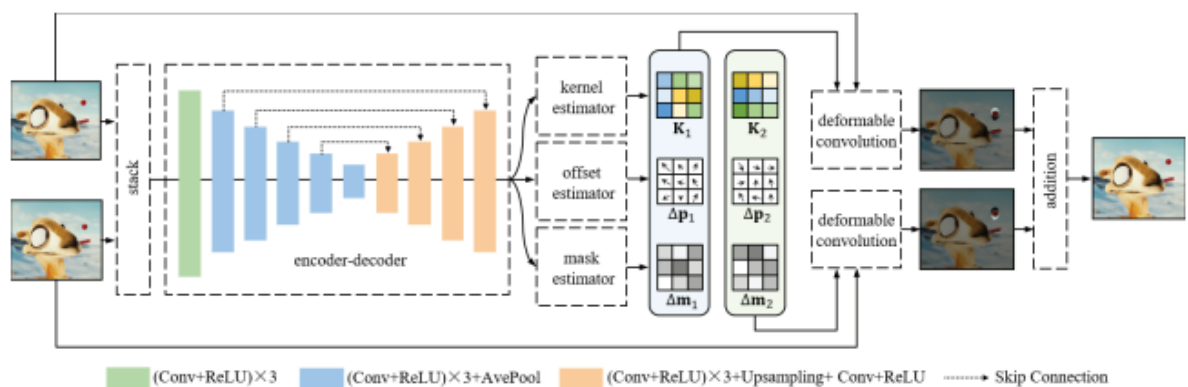


Fonte: CHOI et al (2020)

A arquitetura é totalmente convolucional e organizada em formato *encoder-decoder* com múltiplos blocos residuais e módulos de atenção aplicados nos mapas de características, permitindo à rede focar nos canais mais relevantes durante o processo de interpolação. Essa abordagem contribui para reduzir artefatos em regiões com movimentos complexos ou oclusões, sem a necessidade de estimar movimento explicitamente. Nos experimentos conduzidos pelos autores, o CAIN apresentou resultados comparáveis aos métodos baseados em fluxo óptico, obtendo desempenho competitivo em métricas como PSNR e SSIM, com a vantagem de uma implementação mais eficiente em termos de tempo de inferência.

Em contraposta, XIANHANG e ZHENZHONG (2022) para substituir o uso do fluxo óptico, propuseram uma abordagem denominada *Deformable Separable Convolution* (DSepConv), que de forma adaptável estima os *kernels*, deslocamento e máscaras, permitindo uma seleção criteriosa de pixels relevantes para a imagem interpolada, sendo ideal para vídeos com movimentos bruscos na imagem HUAIZU et al (2018) (Figura 13).

Figura 13 - Arquitetura do modelo XIANHANG e ZHENZHONG



Fonte: XIANHANG e ZHENZHONG (2022)

Entretanto, semelhante a outros métodos de *kernels* adaptativos, essa arquitetura gera somente um *frame* intermediário por vez, não contemplando múltiplos intervalos temporais sem processamento iterativo. Além disso, embora o DSepConv não esteja limitado a janelas fixas, *kernels* muito maiores acarretaria o aumento de custo computacional, o que os autores evitam para manter a eficiência. Nos testes, os resultados reportados utilizando esta técnica indicam ganhos de qualidade (por exemplo, melhoria em torno de 0,9 dB em PSNR sobre o método SepConv) e colocação entre os três melhores métodos no *benchmark Middlebury*, evidenciando que o DSepConv alcança qualidade comparável ou superior ao estado da arte sem depender de fluxo, contexto ou profundidade explícitos.

3.3. Síntese da Literatura Correlata

Os trabalhos analisados abordam a interpolação de *frames* por diferentes estratégias, com o objetivo comum de melhorar a qualidade visual em contextos desafiadores. WENBO et al (2019) e HUAIZU et al (2018) se baseiam fortemente em

fluxo óptico, utilizando profundidade e máscaras de visibilidade, respectivamente, o que permite ganhos em métricas objetivas, mas implica alto custo computacional e limitações em cenários com baixa textura ou objetos distantes.

Por outro lado, SIMON et al (2017) e HYEONGMIN et al (2020) buscaram superar as falhas do fluxo óptico com abordagens baseadas em convoluções adaptativas. O AdaCoF, em particular, avança ao combinar deslocamento e *kernel* em uma única operação, oferecendo mais flexibilidade e robustez em grandes movimentos. No entanto, assim como os demais métodos baseados em *kernels*, ainda produz apenas um quadro por execução e carece de elementos adicionais para aprimorar nitidez.

XIANHANG e ZHENZHONG (2022) propõem uma solução intermediária: ampliam o alcance de movimento por meio de filtros separáveis deformáveis (DSepConv), alcançando melhor resultado sobre métodos anteriores com *kernels* fixos, mas à custa de maior demanda computacional em *offsets* maiores. Já CHOI et al (2020), com o modelo CAIN, eliminam completamente o uso de fluxo óptico e utilizam atenção de canal para refinar a geração de *frames*, reduzindo a complexidade da arquitetura e melhorando a qualidade visual, embora sem explorar explicitamente relações espaciais.

De forma geral, os métodos recentes mostram avanços em fidelidade visual e flexibilidade, mas ainda enfrentam desafios relacionados à escalabilidade, à produção de múltiplos quadros e ao custo de processamento. Esses aspectos reforçam a relevância da proposta deste trabalho, que busca aliar qualidade visual e eficiência computacional por meio de mecanismos de atenção espacial e de canal.

4. ESCOPO

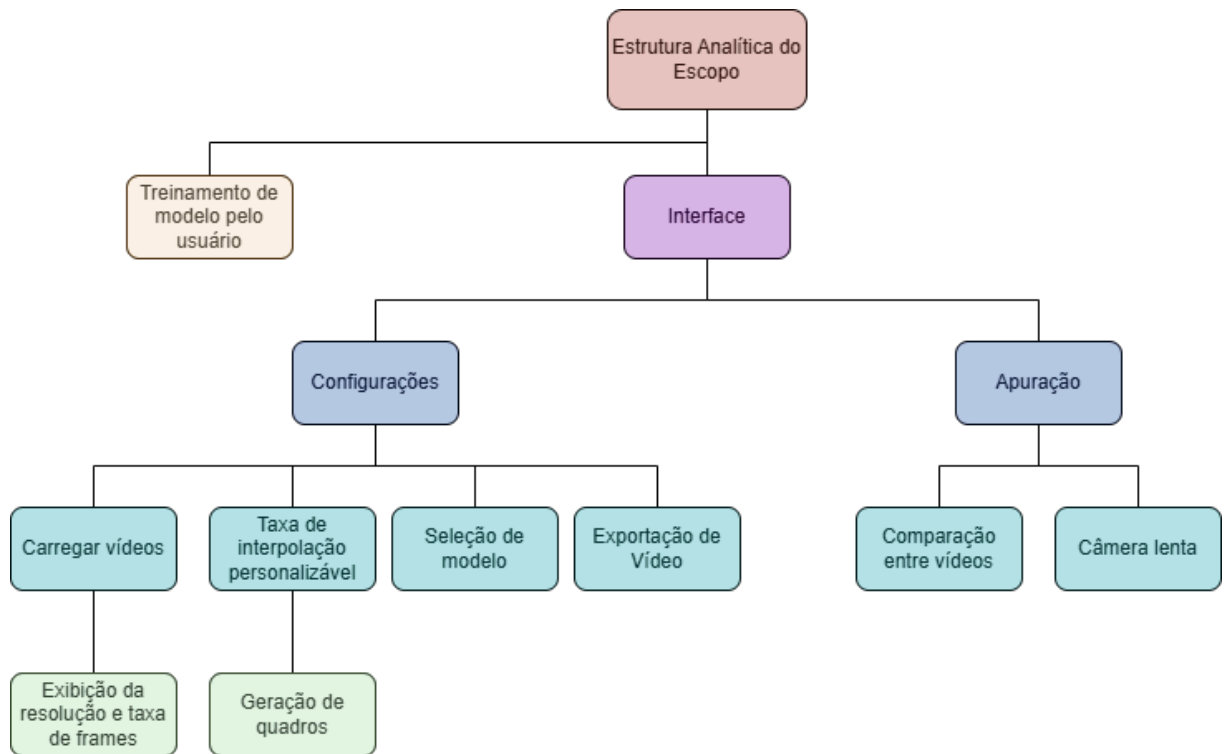
O sistema espera a entrada de vários pares de imagens extraídas a partir de um vídeo fornecido pelo usuário. Cada par é processado de forma sequencial por um modelo, com o objetivo de gerar novos quadros que se integrem naturalmente à sequência original. O sistema é capaz de receber um vídeo completo e realizar automaticamente seu particionamento em pares de quadros consecutivos. Durante essa etapa, o sistema analisa as imagens quanto à resolução e à taxa de *frames* por segundo, a fim de preparar opções de processamento adequadas para as etapas seguintes.

Cada par de quadros são indexados em uma lista de processamento sequencial, permitindo o controle ordenado da execução e evitando a necessidade de intervenção manual. Após a geração dos *frames* intermediários, o sistema reconstrói o vídeo completo, intercalando os *frames* originais com os gerados, de modo a preservar a fluidez da sequência.

Além disso, o sistema fornece uma interface de acompanhamento em tempo real, exibindo o progresso do processamento por meio de uma barra de progresso. Essa interface também apresenta uma estimativa do tempo restante até a conclusão do processo, oferecendo ao usuário maior transparência e controle sobre a execução.

A Figura 14 mostra como é adotado a distribuição hierárquica dos processos definidos pelo escopo.

Figura 14 - Estrutura Analítica do Escopo



Fonte: Os Autores (2025)

4.1.1. Interface

Foi desenvolvida uma interface gráfica autônoma, na qual o usuário poderá interagir com as configurações de processamento, anexar o modelo fornecido no sistema ou treinar um modelo a partir de suas imagens, e por fim anexar um vídeo para realizar a interpolação com as configurações desejadas.

4.1.2. Seleção de Modelo

O protótipo possui a capacidade de anexar o modelo de interpolação de *frames* treinado pelo usuário ou utilizar o modelo de interpolação treinado originalmente neste trabalho.

4.1.3. Treinamento de Modelo Pelo Usuário

Os arquivos de treinamento do modelo são fornecidos com os mesmos hiper-parâmetros de treinamento utilizados no sistema para facilitar a reprodução, caso o usuário deseje treinar o próprio modelo.

4.1.4. Carregar Vídeos

O sistema aceita vídeos nos formatos mais comuns (por exemplo, MP4, AVI, MOV), com resoluções e taxas de quadros por segundo variáveis. As informações a respeito da resolução e taxa de *frames* do vídeo carregado são exibidas na tela.

4.1.5. Geração de Quadros

O mecanismo de geração de quadros intermediários atua em cada par de quadros originais de modo automatizado e sequencial até que todos os pares do vídeo tenham sido interpolados.

4.1.6. Taxa de Interpolação Personalizável

O usuário pode definir a taxa de interpolação, ou seja, quantos quadros intermediários devem ser inseridos entre cada par de quadros originais. Essa configuração permitirá ajustar o nível de suavização do movimento visual conforme a necessidade de cada projeto.

4.1.7. Comparação Entre Vídeos

Após a interpolação de *frames* do vídeo carregado pelo usuário, é possível visualizar simultaneamente o vídeo original e o vídeo com interpolação de *frames* em um *player* com tela dupla, para comparação visual entre os dois vídeos.

4.1.8. Exportação de Vídeo

Na definição de caminho de saída, o sistema possibilita a exportação do vídeo resultante nos mesmos formatos de contêiner suportados na entrada do vídeo. Também é possível definir uma nova resolução para o vídeo interpolado que seja menor ou igual a resolução original.

4.1.9. Câmera Lenta

Foi implementado um mecanismo de câmera lenta, o qual, caso a geração de *frames* por segundo seja maior que 60, a duração do vídeo será estendida para acomodar os novos quadros inseridos.

5. PLANEJAMENTO

A estrutura adotada ao planejamento contempla a escolha das ferramentas tecnológicas, definição de métodos de treinamento e testes, e então a implementação da interface ao nosso modelo predito.

5.1. Tecnologias

A seguir estão apresentadas as tecnologias e abordagem encontradas na literatura utilizadas para a construção do modelo de interpolação de vídeos.

5.1.1. Dados de Treinamento

Para o treinamento e validação do modelo de interpolação de *frames* foi utilizado o *dataset* Vimeo-90k que constitui um conjunto de dados de vídeo. Este dataset é composto por 89.800 videoclipes, extraídos de vídeos originais da plataforma Vimeo. Cada clipe mantém uma resolução padronizada de 448x256 pixels.

É utilizado o subconjunto *triplet dataset for temporal frame interpolation* que possui 73,171 sequências de grupos de 3 *frames* divididos em dados de treino e teste.

O uso de *triplets* permite que o modelo aprenda a prever o *frame* intermediário a partir de seus vizinhos temporalmente adjacentes, capturando dependências contextuais entre o *frame* anterior e o posterior.

5.1.2. Tecnologias de Desenvolvimento

Para o desenvolvimento da solução, escolheu-se Python pela sintaxe clara e pelo vasto ecossistema de bibliotecas voltadas à IA e processamento de imagens. O PyTorch foi adotado como *framework* de *deep learning* pelo estilo imperativo, suporte nativo a GPU e facilidade de prototipação de redes neurais. Para a camada de apresentação, optou-se por PyQt5, que oferece *widgets* prontos e altamente customizáveis, além de integração simples com o *back-end*.

5.1.3. Utilização de Rede Convolucionais

Para a arquitetura principal do modelo, optou-se pelo uso de redes convolucionais em razão de seu forte viés indutivo para dados de imagem (LECUN et al, 1998). As camadas de convolução exploram a localidade espacial e o compartilhamento de parâmetros, reduzindo drasticamente o número total de pesos em comparação com uma rede totalmente conectada. Essa característica facilita a convergência do treinamento mesmo com conjuntos de dados moderados, pois a CNN aprende filtros que capturam padrões como bordas, texturas e movimentos.

5.1.4. Funções de Perda Utilizadas

Para o treinamento do modelo, foram escolhidas três funções de perda visando avaliar o desempenho sob aspectos diferentes. Enquanto uma função pode estagnar em certa situação, outra ainda indicará se há ganho ou piora. Essa complementaridade tende a melhorar a convergência durante o treinamento e a aumentar a capacidade de generalização, pois o modelo recebe múltiplos sinais sobre o que precisa ajustar em seus pesos.

Para a primeira perda foi escolhido o SSIM pelo fato dele avaliar a imagem como um contexto geral, possivelmente auxiliando o modelo a aprender a preservar bordas, texturas e padrões percebidos pelo olho humano, reduzindo borrões e artefatos que poderiam passar despercebidos por perdas puramente pontuais.

Foi optado também pelo *Charbonnier Loss* no qual é um variante suave da perda absoluta (*L1 Loss*), pela sua capacidade de avaliar valores de pixels sem punir o modelo agressivamente por erros, produzindo gradientes estáveis durante o treinamento. Essa característica pode ajudar o modelo a ajustar pesos de forma gradual, evitando oscilações bruscas na otimização.

Por fim, será utilizado um *Perceptual Loss*, que utiliza camadas intermediárias de uma rede VGG19 pré-treinada para comparar recursos de alto nível extraídos dos *frames* de entrada e saída. Em vez de focar apenas na diferença de cor ou intensidade, essa perda pode ajudar o modelo a capturar padrões semânticos, texturas e detalhes.

5.1.5. Atenção Espacial

Foram implementados módulos de Atenção Espacial, que atribuem pesos diferenciados a regiões da imagem conforme sua relevância para a tarefa. Esse mecanismo auxilia o modelo identificar áreas de movimento importantes, concentrando recursos de processamento nesses pontos críticos e reduzindo o ruído em regiões estáticas.

5.1.6. Atenção de Canal

Em conjunto com a atenção espacial, são incluídos módulos de atenção de canal, enfatizando diferentes canais de informação, como padrões de movimento e texturas, conforme sua importância. Assim, uma rede se torna capaz de reforçar os canais mais relevantes para a interpolação e reduz o peso dos menos úteis, melhorando a qualidade dos *frames* gerados durante o processo de interpolação.

5.1.7. Inclusão de Conexões de Atalho - *skip connection*

Foram implementadas conexões de atalho que permitem ao gradiente propagar-se diretamente entre camadas distantes, mitigando o problema de gradiente desaparecido e garantindo que, caso o ajuste residual seja nulo, a representação original seja mantida intacta, possivelmente acelerando o treinamento da rede.

5.1.8. Inclusão de Blocos Residuais

Na rede, são empregados blocos residuais, onde cada bloco irá aprender o resíduo necessário ao processo visando melhorar a capacidade de generalização da rede e mitigar problemas de treinamento como a degradação do desempenho.

5.1.9. Inclusão de Grupos Residuais

Optou-se por estruturar grupos residuais compostos por múltiplos blocos residuais encadeados, conectados via uma ligação à entrada à saída dos grupos, permitindo uma expansão modular e flexível de blocos residuais.

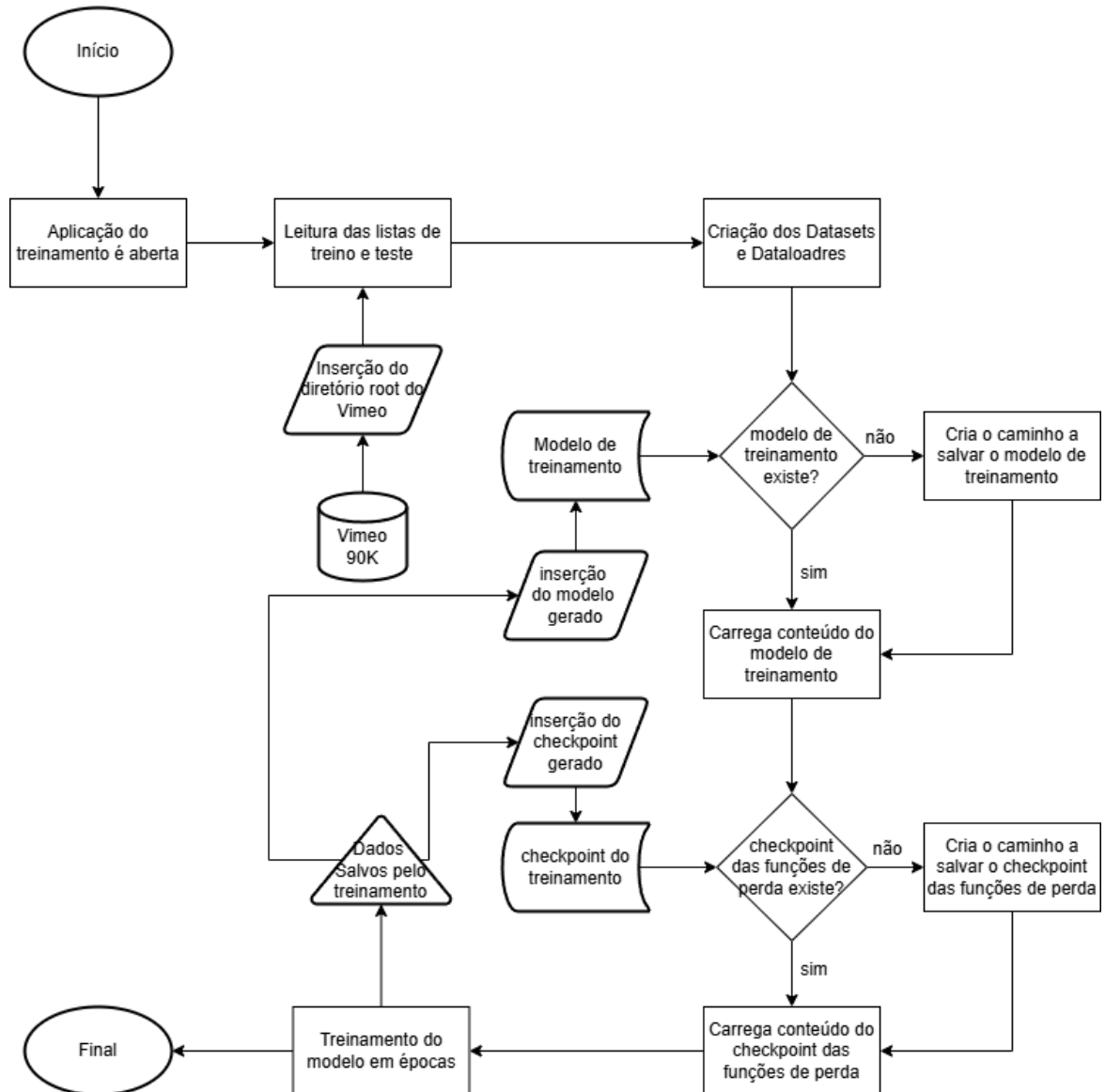
5.1.10. Inclusão de *Pixel Shuffle*

Durante o processo de reconstrução do *frame* intermediário, é aplicado a técnica de *Pixel Shuffle* no lugar de convoluções transpostas para evitar a presença de artefatos *checkerboard* e reduzir o custo computacional já que o *Pixel Shuffle* mantém a maior parte do processamento em mapas de característica de baixa resolução.

5.2. Descrição dos Procedimentos do Sistema

O processo o qual é construída a interpolação de vídeo (Figura 15) toma duas principais dianteiras, sendo a primeira a função de treinamento, que é responsável pela parte técnica de modo a criar o modelo de treinamento importado pela próxima etapa do processo.

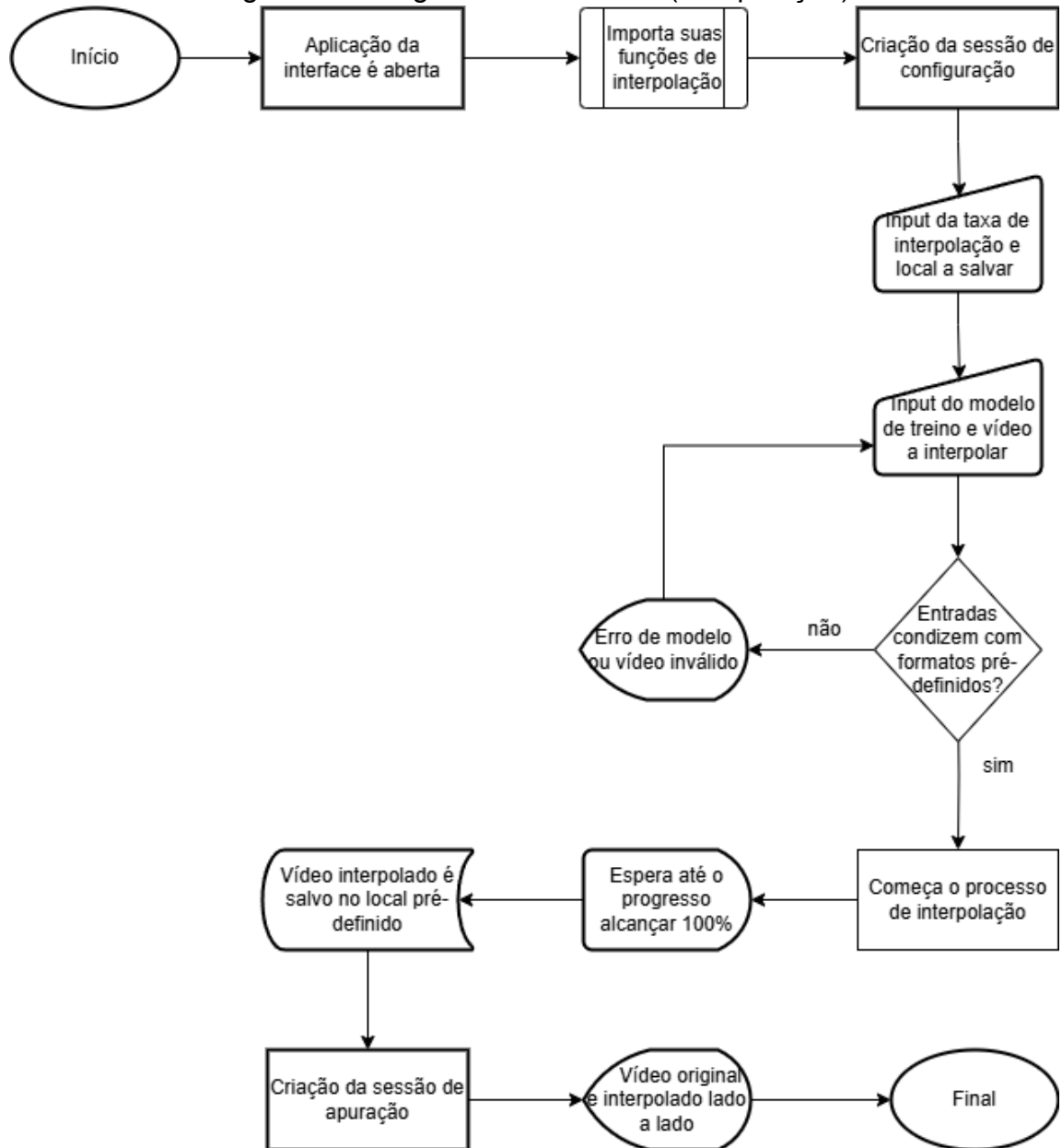
Figura 15 - Diagrama de Processo (Treinamento)



Fonte: Os Autores (2025)

Para a interação do usuário com a aplicação são importadas todas funções de interpolação de vídeo e criação do arquivo de vídeo interpolado junto ao seu áudio original à interface. Tal interface de usuário foi desenvolvida com a biblioteca PyQt5 como API (Interface de Programação de Aplicações), servindo seu propósito com funções facilitadoras para o usuário, assim como o uso de indicadores visuais no caso de confusão pela parte do usuário (Figura 16).

Figura 16 - Diagrama de Processo(Interpolação)



Fonte: Os Autores (2025)

5.3. Protótipo de Telas

O protótipo de telas possui duas abas nas quais o usuário consegue interagir com a aplicação de interpolação de vídeos.

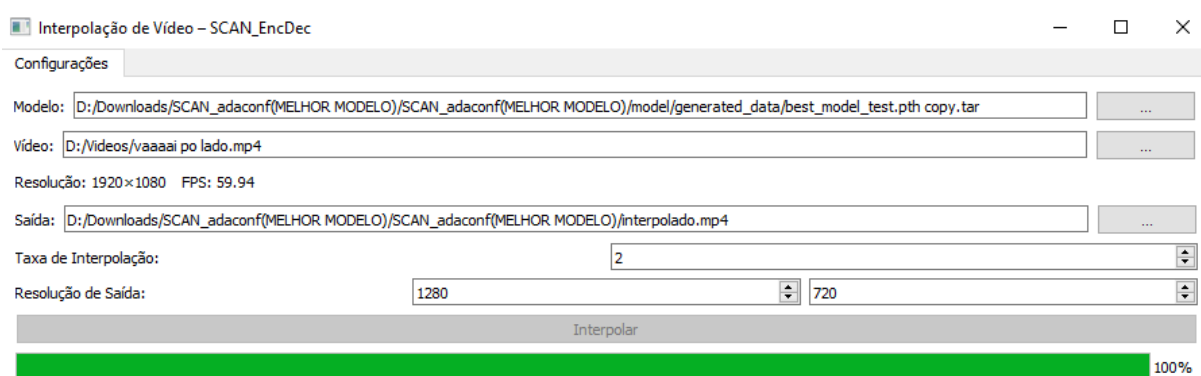
5.3.1. Configurações

Na aba de Configurações ficam todas as funções necessárias para que seja continuado o programa com a criação do vídeo interpolado. Na área de Modelo,

deve-se carregar o modelo de interpolação do computador do usuário, já de maneira compatível ao aceitar apenas extensões do PyTorch. No campo de Vídeo, o usuário deve carregar o vídeo desejado para a interpolação, o qual ao selecionar, informará as informações de resolução e FPS. No local de saída é necessário que insira o caminho da pasta a salvar o vídeo interpolado. No último espaço a colocar informação, o usuário possui a possibilidade de aumentar a taxa de interpolação (Figura 17).

Por fim, ao apertar o botão descrito Interpoliar, o processo de interpolação do vídeo original é iniciado, como também é mostrado o progresso até a interpolação ser finalizada (Figura 17).

[Figura 17 - Janela de configurações de baixa fidelidade](#)

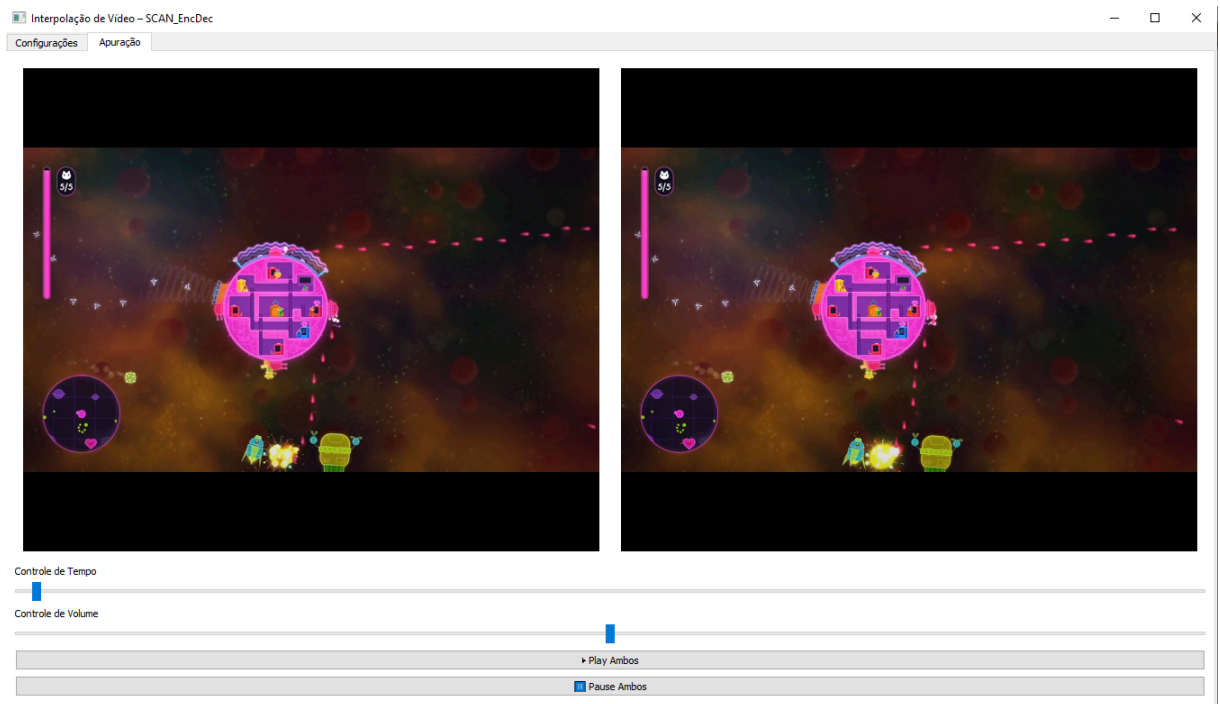


Fonte: Os Autores (2025)

5.3.2. Apuração

Na aba de Apuração são disponibilizados tanto o vídeo original como o vídeo interpolado lado a lado para comparação e praticidade do usuário. Como forma de controle dos vídeos, são utilizados dois *sliders* e dois botões na interface, tendo os *sliders* para controlar o tempo e volume de ambos os vídeos, assim como a opção de reproduzir e pausar ao utilizar os botões dispostos (Figura 18).

Figura 18 - Janela de apuração de baixa fidelidade



Fonte: Os Autores (2025)

6. RESULTADOS

A seção de resultados contempla descrições do que foi desenvolvido como parte do planejamento e arquiteturas construídas ao decorrer do desenvolvimento da solução.

6.1. Resultados do Escopo e Planejamento da Solução

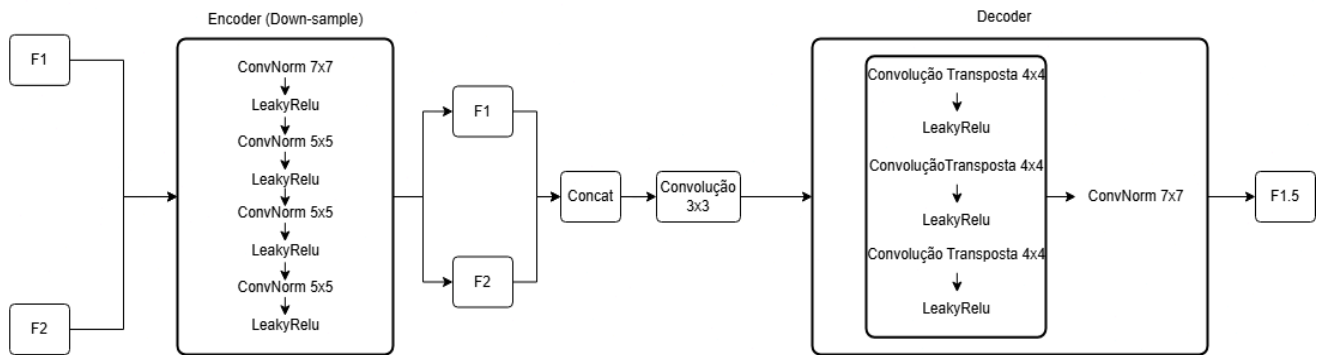
O projeto contempla o desenvolvimento de um sistema de interpolação de *frames* em vídeos, com capacidade de dividir automaticamente um vídeo em pares de quadros consecutivos, processá-los sequencialmente e gerar quadros intermediários que se integrem de forma natural à sequência original. A solução oferece uma interface gráfica que permite ao usuário definir configurações, anexar modelos, além de ajustar a taxa de interpolação desejada. O sistema é compatível com formatos de vídeo MP4, AVI e MOV, exibe dados como resolução e taxa de *frames*, e permite visualizar o vídeo original e o interpolado lado a lado para fins comparativos.

6.2. Arquitetura Inicial

A arquitetura do modelo inicial é formada por uma rede CNN no estilo *encoder-decoder*, composta por quatro camadas no *encoder* e *decoder*. Na primeira camada, ela possui 3 canais de entrada que representam os valores RGB (vermelho, verde e azul) de todos os pixels dos *frames*, logo em seguida, a saída da primeira camada é expandida para 32 canais que são duplicados nas saídas das camadas subsequentes, finalizando com um valor fixo de 192 canais.

No *decoder*, a entrada da primeira camada recebe exatos 192 canais em que é feito o processo inverso ao *encoder*, fatorando os canais pela metade até atingir uma saída fixa de 3 canais na última camada.

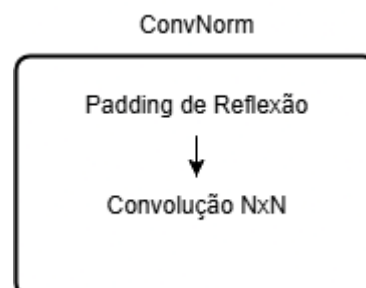
[Figura 19 - Arquitetura do modelo inicial](#)



Fonte: Os Autores (2025)

Como a Figura 19 aponta, os pares de pixels dos *frames* F1(*frame* 1) e F2(*frame* 2) são alimentados ao *encoder* de forma independente. Cada *frame* passa por uma cama inicial de ConvNorm 7x7 com um *stride* igual a 1, que combina *padding* de reflexão e uma convolução NxN, onde N é igual ao valor do *kernel* de ConvNorm (por exemplo, 7x7), seguida de uma função de ativação LeakyRelu para captar informações mais gerais.

[Figura 20 - InvNorm](#)



Fonte: Os Autores (2025)

Para garantir que as operações de convolução não reduzam as dimensões espaciais e nem introduzam artefatos nas bordas, o bloco ConvNorm emprega

primeiro um *padding* de reflexão, que espelha os pixels ao redor de cada borda do *frame* de entrada, garantindo que, ao aplicar uma convolução, exista valores válidos nas vizinhanças exteriores, mantendo a altura e largura inalteradas.

Após a passagem pela primeira camada, aplicam-se três etapas de ConvNorm 5x5 com *stride* igual a 2 seguida por uma função de ativação LeakyRelu. O uso de LeakyRelu visa mitigar um problema denominado *dead ReLU*, em que, quando uma unidade com ativação ReLU recebe sistematicamente entradas negativas, sua derivada se torna zero e ela deixa de atualizar os pesos durante o *backpropagation*, ficando permanentemente inativa. Na LeakyRelu, mesmo que as entradas sejam menores que zero, o gradiente não é completamente anulado, mantendo essas unidades ativas.

Ao final do *encoder*, obtêm-se as *features* individuais de F1 e F2, em que é realizado uma concatenação e fusão de ambos os *frames* por uma convolução 3x3, integrando informações de F1 e F2 em um único mapa de *features*, como demonstrado na Figura 19.

No *decoder*, as *features* fundidas passam por três camadas de convolução transposta 4x4 com *stride* igual a 2 intercaladas com LeakyRelu, as quais recuperam progressivamente a resolução espacial enquanto reduzem o número de canais. Por fim, uma última ConvNorm 7x7 gera o *frame* interpolado “F1.5”.

6.3. Apuração do Modelo Inicial

O modelo inicial foi treinado por 150 épocas com uma taxa de aprendizado fixa de $3e-5$ (0.00003), resultando em uma acurácia de 86,85% e 29,17 dB nas métricas SSIM e PSNR, respectivamente na porção de validação do *dataset* (Figura 19).

[Figura 21 - Resultado da Interpolação](#)

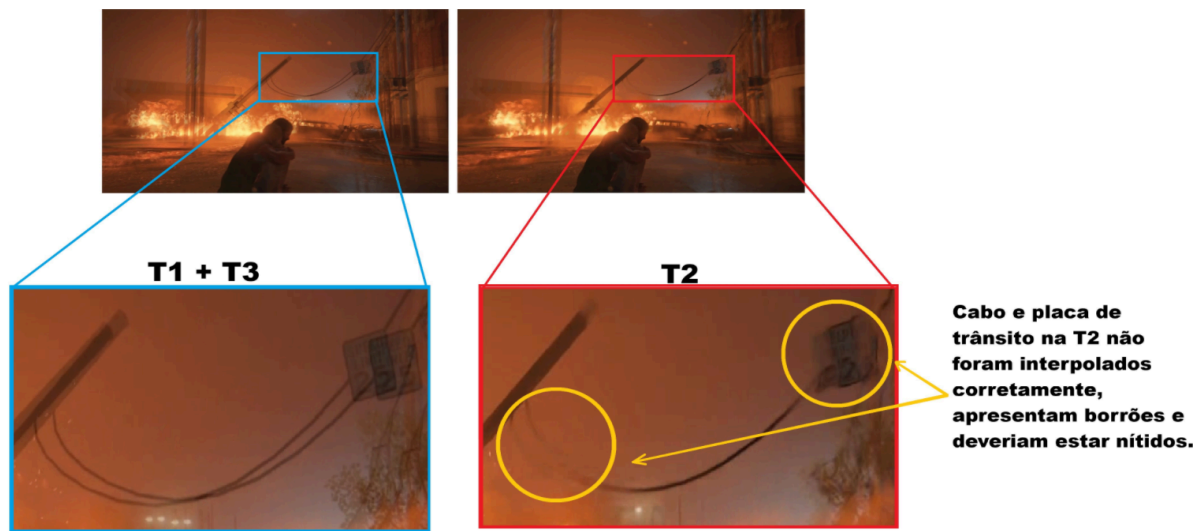


Fonte: Os Autores (2025)

Conforme ilustrado na Figura 21, foram concatenados três *frames* que possuem sequência direta da esquerda para a direita nos intervalos T_1 , T_2 e T_3 respectivamente, em que o *frames* mais a esquerda (T_1) e direita (T_3) são os *frames* originais do vídeo e o *frame* intermediário (T_2) é o conteúdo predito pelo modelo. Percebe-se que o modelo aprendeu cores, contraste, iluminação e a composição geral para formular uma imagem intermediária.

Apesar do modelo ter aprendido os principais elementos que compõem os *frames* T_1 e T_3 , o mesmo passa por dificuldades para interpolar corretamente movimentos em objetos pequenos ou em elementos que passam por uma mudança brusca de posição (Figura 22).

Figura 22 - Close-up dos resultados



Fonte: Os Autores (2025)

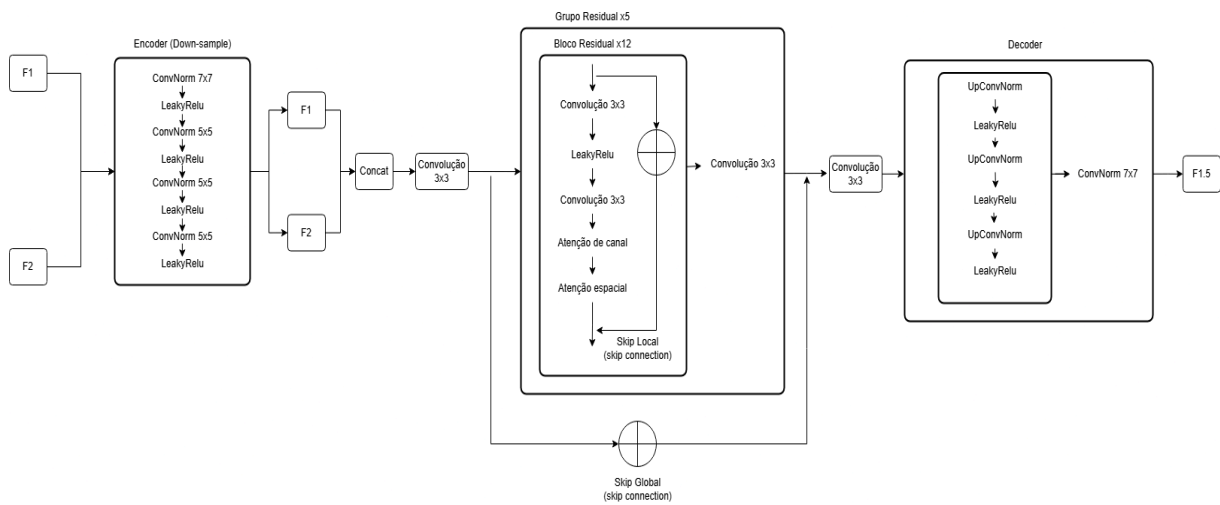
Na Figura 22, foi destacada uma porção dos *frames* $T_1 + T_3$, à esquerda, e T_2 , à direita, em que se nota um poste caindo, com uma placa de trânsito anexada a um fio. Ao analisar e comparar ambos destaques, percebe-se que o modelo não foi capaz de interpolar corretamente o movimento do fio e da placa de trânsito, gerando borrões visuais no *frame* predito T_2 , como apontado na Figura 22.

Na eventualidade do modelo ter interpolado os elementos corretamente, esperava-se a ausência de borrões, nitidez clara do fio conectado ao poste e dos elementos escritos na placa de trânsito.

6.4. Arquitetura Final

A arquitetura sofreu uma expansão em seu *pipeline* de processamento, implementando as tecnologias mencionadas anteriormente, com o objetivo de auxiliar o modelo a interpolar *frames* com maior precisão (Figura 23).

Figura 23 - Arquitetura do modelo final



Fonte: Os Autores (2025)

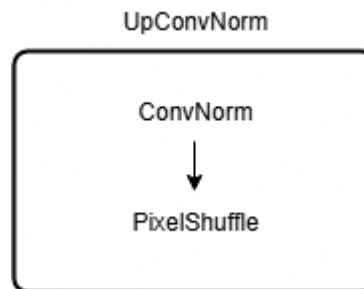
Como a Figura 23 aponta, foi implementado uma etapa adicional durante o processo de convolução. Entre a etapa de *encoder* e *decoder*, existem 5 grupos residuais, onde as *features* são salvas previamente à entrada de cada grupo e então são somadas à saída de cada grupo, representado pelo símbolo " $\oplus$ ", preservando as *features* originais (*skip connection*).

Dentro de cada grupo também existem 12 blocos residuais, onde, similar aos grupos residuais, é efetuado um *skip connection* para a saída de cada bloco. Também, em cada bloco existe uma convolução 3x3, seguido de uma função de ativação LeakyRelu e então outra convolução 3x3, dando continuidade ao processamento do bloco através da passagem das *features* por um módulo de atenção de canal e atenção espacial, que procuram reforçar canais mais relevantes para a interpolação e destacando as regiões importantes da imagem, respectivamente.

Em outras palavras, este processamento adicional pode ser interpretado como uma etapa de refinamento das *features* extraídas dos *frames* para auxiliar o modelo em inferir uma informação intermediária com maior precisão.

Na etapa do *decoder*, foram efetuadas modificações para substituir a convolução transposta por *pixel shuffle*, que evita artefatos de tipo *checkerboard* e reduz o custo computacional durante o processo de reconstrução do *frame* intermediário.

[Figura 24 - UpConvNorm](#)



Fonte: Os Autores (2025)

Cada camada composta por convolução transposta foi efetivamente substituída por um módulo denominado UpConvNorm. Como ilustrado na Figura 24, este módulo é composto por outro módulo ConvNorm, que mantém a altura e largura da imagem de entrada durante o processo de convolução, seguido por um *pixel shuffle*, evitando os possíveis vies pela utilização de convoluções transpostas.

6.5. Apuração do Modelo Final

O modelo final foi treinado por um total de 500 épocas, onde nas épocas 0-399 foi utilizado a mesma taxa de aprendizado ($3e-5$) e das épocas 400-500 a mesma é reduzida para 0.00001 ($1e-5$), resultando em um incremento de 8,89% e 5,34 db na porção de validação do *dataset*, totalizando 95,74% e 34,51 db nas métricas SSIM e PSNR, respectivamente.

[Figura 25 - Comparativo entre modelos](#)



Fonte: Os Autores (2025)

Na Figura 25, foram concatenados dois *frames*. A imagem mais esquerda representa a informação predita pelo modelo inicial (*Frame Modelo Inicial*) e a imagem a direita exibe a informação predita pelo modelo final (*Frame Modelo Final*).

Para este comparativo, foram utilizados os mesmos *frames* T_1 e T_3 ilustrados previamente na Figura 21 para gerar o *Frame Modelo Final*. Nota-se que o modelo final aparenta estar produzindo *frames* com maior precisão, reforçado pelo fato de elementos na imagem *Frame Modelo Inicial* estarem desfocados devido a presença de *ghosting*, enquanto este problema foi consideravelmente mitigado no modelo final, mantendo a nitidez dos elementos presentes na imagem.

Figura 26 - Close-up dos modelos



Fonte: Os Autores (2025)

Retornando aos problemas do modelo inicial, o mesmo não era capaz de interpolar elementos com uma nitidez que fosse visualmente satisfatória. Similar a Figura 22, na Figura 26 foram aproximadas as mesmas regiões utilizadas anteriormente para exibir os resultados de interpolação. Ao lado esquerdo encontra-se o destaque do *Frame Modelo Inicial* e ao lado direito está o *Frame Modelo Final*. Percebe-se que o modelo final foi capaz de interpolar o fio conectado ao poste e dos elementos escritos na placa de trânsito com nitidez, onde a mesma tarefa havia falhado no modelo inicial.

Entende-se então que após a expansão, avaliação e modificação da arquitetura, a precisão de interpolação do modelo foi ampliada, permitindo que o modelo seja capaz de inferir *frames* com menos artefatos visuais.

6.6. Interface de Alta Fidelidade

Na interface de alta fidelidade, na aba de configurações, é possível definir os principais parâmetros que são usados pelo sistema de interpolação, sendo possível selecionar o modelo, o vídeo que será interpolado, a taxa de interpolação, o diretório de saída e resolução do vídeo interpolado (Figura 27).

Figura 27 - Interface de configuração

Interpolação de Vídeo – SCAN_EncDec

Configurações

Modelo
Selecione o modelo de interpolação que será usado para interpolar o vídeo.

Arraste ou clique para selecionar o modelo (.pth, .tar)

Vídeo de Entrada
Selecione o vídeo que será interpolado.

Arraste ou clique para selecionar o vídeo (.mp4, .avi, .mov)

Resolução: — FPS: —

Taxa de Interpolação: 2

Diretório de Saída
Selecione o diretório em que o vídeo interpolado será enviado.

Diretório Selecionar Diretório

Resolução de Saída: 1 × 1

Interpol

Fonte: Os Autores (2025)

A interface principal é dividida em três seções principais:

Na seção Modelo (Figura 28), o usuário deve carregar um modelo de rede neural treinado para realizar a interpolação. O botão azul permite selecionar arquivos nos formatos .pth ou .tar, que normalmente armazenam pesos de modelos em *frameworks* como PyTorch.

[Figura 28 - Seção do modelo](#)

Modelo

Selecione o modelo de interpolação que será usado para interpolar o vídeo.

Arraste ou clique para selecionar o modelo (.pth, .tar)

Fonte: Os Autores (2025)

Na sessão Vídeo (Figura 29), o usuário seleciona o vídeo que será interpolado. São aceitos arquivos nos formatos .mp4, .avi e .mov. A interface exibe campos informativos sobre a resolução e o FPS do vídeo após o carregamento, além de permitir ao usuário definir a taxa de interpolação por meio de um controle de valor numérico (*spinner*). Por exemplo, uma taxa de interpolação 2 duplicará o número de quadros entre os quadros originais.

[Figura 29 - Seção do vídeo de entrada](#)

Vídeo de Entrada

Selecione o vídeo que será interpolado.

Arraste ou clique para selecionar o vídeo (.mp4, .avi, .mov)

Resolução: — FPS: —

Taxa de Interpolação:

2

Fonte: Os Autores (2025)

A seção Diretório de Saída (Figura 30) permite ao usuário escolher onde o vídeo interpolado será salvo. Através do botão "Selecionar Diretório", um caminho de saída é definido. Também é possível ajustar a resolução de saída, com controles numéricos para definir o multiplicador de escala.

[Figura 30 - Seção do diretório de saída](#)

Diretório de Saída

Selecione o diretório em que o vídeo interpolado será enviado.

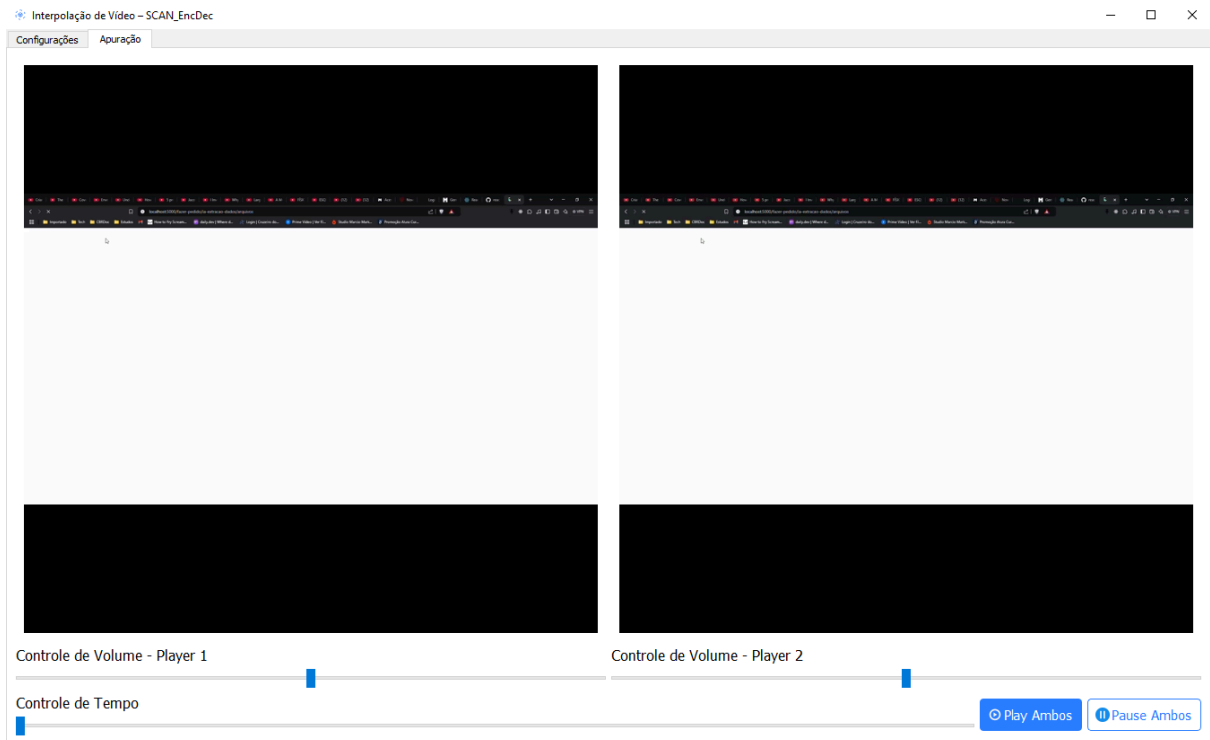
Selecionar Diretório

Resolução de Saída: ×

Fonte: Os Autores (2025)

A aba Apuração (Figura 31) tem como finalidade principal permitir a comparação simultânea entre o vídeo original e o vídeo interpolado. A visualização lado a lado dos dois vídeos facilita a análise qualitativa dos resultados, permitindo ao usuário observar com clareza as melhorias visuais proporcionadas pela interpolação, como o aumento da fluidez dos movimentos.

[Figura 31 - Aba de Apuração](#)



Fonte: Os Autores (2025)

Na parte superior da tela estão os dois players de vídeo. O da esquerda reproduz o vídeo original, enquanto o da direita exibe o vídeo gerado após a interpolação. Ambos são exibidos com a mesma escala visual, o que permite uma comparação direta dos quadros correspondentes. A interface garante que os dois vídeos sejam sincronizados no tempo, de modo que suas execuções possam ser iniciadas e pausadas simultaneamente.

Na parte inferior da janela encontram-se os controles de volume individuais para cada player, bem como um controle deslizante que ajusta o tempo de reprodução de ambos os vídeos de forma sincronizada. Ainda nessa área, há dois botões: um que permite iniciar a reprodução conjunta dos vídeos e outro que os pausa simultaneamente. Esses recursos facilitam a identificação visual de diferenças sutis entre os vídeos, sendo especialmente úteis na avaliação de artefatos visuais, falhas de interpolação ou na verificação da suavização de movimentos.

Do ponto de vista funcional, essa tela tem papel fundamental na validação prática dos modelos utilizados no sistema. A comparação visual direta permite ao usuário não apenas perceber as melhorias introduzidas pelo modelo de interpolação, mas também identificar limitações e oportunidades de refinamento. Por isso, ela é uma parte essencial do processo de desenvolvimento e avaliação do sistema.

7. CONCLUSÃO

Com base na exposição inicial deste trabalho, foi possível contextualizar o problema da interpolação de *frames* em vídeos, destacando as limitações dos métodos tradicionais baseados em fluxo óptico, como sua alta complexidade computacional devido a utilização de modelos de IA e sensibilidade a condições adversas. A justificativa reforçou a relevância de investigar abordagens que eliminem a necessidade do fluxo óptico por técnicas mais leves e robustas.

A proposta apresentada busca contornar as dificuldades técnicas dos métodos baseados em fluxo óptico e também oferecer uma solução mais acessível a contextos diversos, com foco na preservação da fidelidade visual. Os objetivos gerais e específicos definiram o escopo da solução, abrangendo desde o treinamento do modelo até sua validação por métricas objetivas e perceptuais, como PSNR e SSIM.

Para a escolha das funções de perda e dos mecanismos de atenção, reconhece-se a complexidade diante dos conceitos relativos à visão computacional no uso de abordagens diferentes para cada trabalho correlato analisado.

No mesmo contexto, as funções de perda mostraram-se elementos importantes para o treinamento dos modelos analisados. Entre as mais relevantes estão:

- SSIM, que mede a semelhança entre dois *frames*;
- *Charbonnier Loss*, que torna o modelo mais robusto a valores discrepantes;
- *Perceptual Loss*, que busca capturar padrões de textura e detalhes visuais.

Durante o desenvolvimento do modelo proposto, foram incorporadas modificações na arquitetura base que influenciaram diretamente o desempenho obtido. Entre essas alterações, destacam-se os módulos de atenção espacial e atenção de canal, responsáveis por direcionar a rede a priorizar regiões mais relevantes do quadro, preservando detalhes e reduzindo borramentos em áreas com

movimento. O uso de grupos e blocos residuais contribuiu para melhorar a extração e reutilização de características, otimizando a profundidade da rede sem aumentar excessivamente a complexidade.

Adicionalmente, a aplicação de ConvNorm combinada com UpConvNorm e *PixelShuffle* no processo de codificação e decodificação auxiliou na estabilização do treinamento e na minimização da perda de informações entre etapas, resultando em frames intermediários mais nítidos e com melhor definição de bordas.

Essas adaptações refletiram-se nos resultados obtidos, com valores de 34,86 dB (era 29,17 dB) para PSNR e 95,74% (era 86,85%) para SSIM, indicando melhorias quantitativas em relação à preservação de detalhes e à similaridade estrutural entre os quadros originais e os interpolados. Na análise qualitativa, observou-se redução de artefatos e transições mais suaves entre frames, reforçando que as modificações arquiteturais implementadas contribuíram para o aumento da robustez do sistema frente a variações de movimento e presença de oclusões.

7.1. Desafios no Desenvolvimento

Durante o desenvolvimento do modelo, um dos principais desafios que se mostrou imediatamente aparente foi o balanceamento entre carga computacional e complexidade do modelo. Por um lado, aumentar a quantidade de canais e camadas da arquitetura pode aumentar a capacidade do modelo de compreender padrões mais complexos como texturas, detalhes e movimentos. Porém, isso significa também um aumento de tempo de treinamento e, consequentemente, o de inferência para o usuário final. Isso se torna extremamente desafiador, levando em consideração que o modelo deve possuir a capacidade para aprender o movimento entre dois *frames* e também compreender outros aspectos que compõem uma imagem, como cores, contraste, iluminação e composição geral.

Outra dificuldade no desenvolvimento do modelo inicial foi a definição dos parâmetros de treinamento, como taxa de aprendizado, funções de perda e número de épocas. Taxas de aprendizado elevadas (por exemplo, 0.001), mostraram uma aceleração na convergência do modelo, porém elas apresentaram dificuldades em melhorar a acurácia do modelo em épocas altas (> 100). Por outro lado, taxas mais

baixas favorecem ganhos de precisão em fases finais, porém demandam um custo de tempo substancialmente maior para atingir a convergência.

Para garantir uma comparação justa entre diferentes versões do modelo, foi necessário treinar o modelo do zero para cada nova implementação de técnicas ou alteração de parâmetros (por exemplo, troca de função de perda, incremento ou decremento da taxa de aprendizado). Essa necessidade de recriar todo o histórico de aprendizado para cada nova configuração resultava em longos períodos de espera antes de se avaliar se uma mudança gerava ganho significativo ou deveria ser descartada.

Em contrapartida, a utilização do PyTorch, como biblioteca principal para a arquitetura do modelo, facilitou a estruturação, entendimento e prototipação de modelos para a realização de testes.

Durante a implementação dos grupos e blocos residuais, que também são compostos pelos módulos de atenção espacial e atenção de canal, encontrar a quantidade ideal de grupos e blocos se mostrou um desafio devido ao universo de possíveis combinações destes módulos, já que uma quantidade pequena de blocos e grupos não gera um incremento justo de performance, e inversamente, ampliar a quantidade dos mesmos elementos poderia desnecessariamente aumentar o custo computacional devido às operações convolucionais extras adicionadas, ou seja, um caso de retornos decrescentes.

Apesar disso, a modularização deste processo de refinamento facilitou a busca destes parâmetros ideais durante o processo de treinamento e testes.

Para a implementação da interface, a utilização da biblioteca PyQt5 facilitou a criação da interface de baixa fidelidade, integração com o *back-end* e também permitiu a integração com CSS (*Cascading Style Sheets*), comumente utilizado em sites, permitindo a customização da interface de alta fidelidade e tornando-a mais amigável e atrativa para o usuário.

Embora a biblioteca PyQt5 tenha facilitado a criação da interface inicial, houve algumas dificuldades em relação à seu uso para a elaboração da interface de

alta fidelidade, a maior delas sendo a documentação incompleta e não intuitiva, que dificultou o desenvolvimento de uma interface mais complexa, e também, a integração bastante limitada com CSS 2 (*Cascading Style Sheets versão 2*), que é uma versão deveras antiga e ultrapassada da linguagem supracitada, tendo vários recursos indisponíveis.

7.2. Trabalhos Futuros

Pode-se falar do uso de GAN's (*generative adversarial network*) como uma outra ideia de rede neural, o qual possui duas redes de treinamentos que competem entre si para dados mais autênticos. Ainda sobre o modelo, seria possível utilizar o método de treino aluno-professor (*student-teacher*), que em suma utiliza de duas redes neurais, uma de alta e outra de baixa complexidade com as mesmas características da complexa, a fim de obter resultados utilizando menos poder computacional

Em quesito a funções de perda do modelo, outras podem ser atribuídas para dar mais peso a detalhes mais finos, como ETM (*Edge Texture Metric*) que foca em medir a fidelidade de texturas e bordas entre o frame original e o interpolado, penalizando o modelo mesmo quando a similaridade global se mostre decente. Outra possível alternativa viável é a ARL (*Adaptive Robust Loss*) que como a *Charbonnier Loss* já utilizada, é uma função de perda robusta e sensível a valores discrepantes com o adicional de ser mais flexível a adaptação ao mudar de forma ao longo dos dados recebidos na época de treinamento.

7.3. Considerações Finais

O desenvolvimento deste trabalho permitiu compreender as limitações e potencialidades dos métodos de interpolação de frames em vídeos, especialmente no que diz respeito ao uso de fluxo óptico. Observou-se que, embora o fluxo óptico seja amplamente empregado, seu alto custo computacional e baixa eficácia em cenários com movimentos abruptos ou oclusões justificam a busca por alternativas mais leves e robustas.

O modelo final foi treinado por 500 épocas, utilizando uma taxa de aprendizado de $3e-5$ até a época 399 e $1e-5$ entre as épocas 400 e 500. Esse ajuste

resultou em um incremento de 8,89% no SSIM e 5,34 dB no PSNR na validação, alcançando 95,74% e 34,51 dB, respectivamente. A comparação entre o modelo inicial e o final evidencia uma redução significativa de artefatos, como *ghosting*, e maior nitidez em detalhes finos, como fios e sinalizações, demonstrando que as modificações na arquitetura aumentaram a precisão de interpolação e a capacidade do modelo em gerar *frames* com maior fidelidade visual.

Durante o processo de implementação, destacaram-se desafios relevantes, como o equilíbrio entre desempenho e custo computacional, a escolha adequada de hiperparâmetros e a necessidade de retreinar o modelo a cada alteração significativa. Ainda assim, o uso de *frameworks* como PyTorch e PyQt5 possibilitou maior agilidade no desenvolvimento e integração entre as etapas de *back-end* e interface de usuário, mesmo diante de limitações encontradas na personalização da camada visual.

Em síntese, o presente trabalho contribui não apenas com uma solução técnica viável para o problema investigado, mas também com um registro detalhado das etapas, escolhas e desafios enfrentados, podendo servir de base para futuros projetos e aprimoramentos na área.

REFERÊNCIAS (alfabética)

BARRON, J. T. **A General and Adaptive Robust Loss Function**. arXiv preprint arXiv:1701.03077, 2017. Disponível em: <https://arxiv.org/abs/1701.03077>. Acesso em: 03 mai. 2025.

BRIEDIS, K. M. et al. **Neural Frame Interpolation for Rendered Content**. ACM Trans. Graph., Vol. 40, No. 6, Article 239, 2021. Disponível em: <https://studios.disneyresearch.com/app/uploads/2021/10/Neural-Frame-Interpolation-for-Rendered-Content.pdf>. Acesso em: 06 abr. 2025.

CHOI, M.; KIM, H.; HAN, B.; XU, N.; LEE, K. M. **Channel Attention Is All You Need for Video Frame Interpolation**. Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence (AAAI-20), Vancouver, Canada, 2020. Disponível em: <https://cdn.aaai.org/ojs/6693/6693-13-9922-1-10-20200521.pdf>. Acesso em: 02 mai. 2025.

CLOUARD, R. **Peak Signal-to-Noise Ratio (PSNR)**. Pandore Documentation, 2023 Disponível em: <https://clouard.users.greyc.fr/Pandore/programmes/en/operatorsP0/ppsnr.html#:~:text=Because%20many%20signals%20have%20a.between%2030%20and%2040%20dB>. Acesso em: 16 mai. 2025.

DAI, J. et al. **Deformable Convolutional Networks**. ICCV, 2017. Disponível em: <https://arxiv.org/abs/1703.06211>. Acesso em: 02 mai. 2025.

DEGUZMAN, D. **The power of video in news media: how increased audience engagement drives success**. The Associated Press, 2024. Disponível em: <https://www.ap.org/insights/the-power-of-video-in-news-media-how-increased-audience-engagement-drives-success/>. Acesso em: 06 abr. 2025.

DEQUING, S; XIAONDONG, Y; MING-YU, L; JAN, K. PWC-Net: **CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume**. ArXiv, 2018. Disponível em: <https://arxiv.org/abs/1709.02371>. Acesso em: 06 abr. 2025.

FARNEBÄCK, G; BIGUN, J; GUSTAVSSON, T. **Two-Frame Motion Estimation Based on Polynomial Expansion**. Image Analysis, 2003. Disponível em: https://doi.org/10.1007/3-540-45103-X_50. Acesso em: 06 abr. 2025

FISCHER, P; ALEXEY, D et al. **FlowNet: Learning Optical Flow with Convolutional Networks**. ArXiv, 2015. Disponível em: <https://arxiv.org/abs/1504.06852>. Acesso em: 06 abr. 2025.

FIVEABLE. **Peak Signal-to-Noise Ratio (PSNR)**. Biomedical Engineering II – Key Terms, 2025. Disponível em: <https://fiveable.me/key-terms/biomedical-engineering-ii/peak-signal-to-noise-ratio-psnr>. Acesso em: 16 mai. 2025.

FLYNN, J.R. et al. **Image Quality Assessment Using the SSIM and the Just Noticeable Difference Paradigm**, 2013. Disponível em: https://link.springer.com/content/pdf/10.1007/978-3-642-39360-0_3.pdf. Acesso em: 20 set. 2025.

GEEKSFORGEEKS. **What is a Pixel?**, 2024. Disponível em: <https://www.geeksforgeeks.org/what-is-a-pixel/>. Acesso em: 02 mai. 2025.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep Learning**. MIT Press, 2016. Disponível em: <https://www.deeplearningbook.org/>. Acesso em: 02 mai. 2025.

HAYKIN, S. **Neural Networks and Learning Machines**. 3ª ed., Pearson, 2009. Disponível em: <http://dai.fmph.uniba.sk/courses/NN/haykin.neural-networks.3ed.2009.pdf>. Acesso em: 02 mai. 2025.

HE, K. et al. **Deep Residual Learning for Image Recognition**. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016. Disponível em: <https://arxiv.org/abs/1512.03385>. Acesso em: 14 mai. 2025.

HENRYRU. **The concepts of Deep Learning, Neural Networks, Linear regression**. Medium, 12 jun. 2022. Disponível em:

https://medium.com/@cheerful_whip_wombat_910/the-concepts-of-deep-learning-472ff58488bb. Acesso em: 05 maio 2025.

HU, J.; SHEN, L.; SUN, G. **Squeeze-and-Excitation Networks**. CVPR, 2018. Disponível em: https://openaccess.thecvf.com/content_cvpr_2018/papers/Hu_Squeeze-and-Excitation_Networks_CVPR_2018_paper.pdf. Acesso em: 02 mai. 2025.

HUAIZU, J. et al. 2018. **Super slomo: High quality estimation of multiple intermediate frames for video interpolation**. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 9000–9008. Disponível em: <https://arxiv.org/abs/1712.00080>. Acesso em: 02 mai. 2025.

HYEONGMIN, L. et al. 2020. **AdaCoF: Adaptive collaboration of flows for video frame interpolation**. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 5316–5325. Disponível em: <https://arxiv.org/abs/1907.10244>. Acesso em: 02 mai. 2025.

JOHNSON, J. et al. **Perceptual Losses for Real-Time Style Transfer and Super-Resolution**. ECCV, 2016. Disponível em: <https://cs.stanford.edu/people/jcjohns/papers/eccv16/JohnsonECCV16.pdf>. Acesso em: 02 mai. 2025.

KIM, J.; LEE, C. **Reliable Perceptual Loss Computation for GAN-Based Super-Resolution With Edge Texture Metric**. ResearchGate, ago. 2021. Disponível em: https://www.researchgate.net/figure/Some-feature-maps-extracted-from-a-pre-trained-VGG-19-feature-extractor-of-the-DIV2K-0312_fig1_354174248. Acesso em: 05 maio 2025.

LECUN, Y. et al. **Gradient-Based Learning Applied to Document Recognition**. Proceedings of the IEEE, v. 86, n. 11, p. 2278–2324, 1998. Disponível em: <https://ieeexplore.ieee.org/document/726791>. Acesso em: 02 mai. 2025.

LEDIG, C.; THEIS, L.; HUSZÁR, F.; CABALLERO, J.; CUNNINGHAM, A.; ACOSTA, A.; AITKEN, A.; TEJANI, A.; TOTZ, J.; WANG, Z.; SHI, W. **Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network**. arXiv preprint

arXiv:1609.04802, 2017. Disponível em: <https://arxiv.org/pdf/1609.04802v5>. Acesso em: 02 mai. 2025.

MACKIN, A; ZHANG ,F; BULL, D. R. **A study of subjective video quality at various frame rates**. IEEE INTERNATIONAL CONFERENCE ON IMAGE PROCESSING (ICIP), 2015, Quebec City. Disponível em: <https://scispace.com/pdf/a-study-of-subjective-video-quality-at-various-frame-rates-hx11d80oi0.pdf>. Acesso em: 02 mai. 2025.

MILVUS. **What is a Patch in Image Processing?** AI Quick Reference, 2025 Disponível em: <https://milvus.io/ai-quick-reference/what-is-a-patch-in-image-processing>. Acesso em: 16 mai. 2025.

MOSTAFA. **Depth map artefacts reduction: a review**. IET Image Process, 2020. Disponível em: https://www.researchgate.net/figure/Result-of-35-compared-to-90-and-49-for-filling-hole-pixels-and-filtering-the_fig2_341404281. Acesso em: 02 mai. 2025.

SHI, W.; CABALLERO, J.; HUSZÁR, F.; TOTZ, J.; AITKEN, A.; BISHOP, R.; RUECKERT, D.; WANG, Z. **Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network**. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016. Disponível em: <https://arxiv.org/abs/1609.05158>. Acesso em: 14 mai. 2025.

SIMON, N. et al. 2017. **Video frame interpolation via adaptive convolution**. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 670–679. Disponível em: <https://arxiv.org/abs/1703.07514>. Acesso em: 02 mai. 2025.

SPARACELLO, Vitale. **A gentle introduction to Geometric Deep Learning**. Medium, 22 mai. 2022. Disponível em: <https://medium.com/dataroots/a-gentle-introduction-to-geometric-deep-learning-by-vitale-sparacello-91e529cf439b>. Acesso em: 05 maio 2025.

WANG, Z. et al. **Image Quality Assessment: From Error Visibility to Structural Similarity**. IEEE Transactions on Image Processing, v. 13, n. 4, p. 600–612, 2004.

Disponível em: <https://www.cns.nyu.edu/pub/eero/wang03-reprint.pdf>. Acesso em: 02 mai. 2025.

WENBO, B. et al. 2019. **Depth-aware video frame interpolation**. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 3703–3712. Disponível em: <https://arxiv.org/abs/1904.00830>. Acesso em: 02 mai. 2025.

WOO, S. et al. CBAM: **Convolutional Block Attention Module**. ECCV, 2018. Disponível em: https://openaccess.thecvf.com/content_ECCV_2018/papers/Sanghyun_Woo_Convolutional_Block_Attention_ECCV_2018_paper.pdf. Acesso em: 02 mai. 2025.

XIANHANG, C.; ZHENZHONG, C. 2020. **Video frame interpolation via deformable separable convolution**. In Proceedings of the AAAI Conference on Artificial Intelligence, v. 34, p. 10607–10614. Disponível em: <https://ojs.aaai.org/index.php/AAAI/article/view/6634>. Acesso em: 23 mai. 2025.

ZACHARY T; JIA D. **RAFT: Recurrent All-Pairs Field Transforms for Optical Flow**. ArXiv, 2020. Disponível em: <https://arxiv.org/abs/2003.12039>. Acesso em: 06 abr. 2025.

ZHANG, Y. et al. **Image Super-Resolution Using Very Deep Residual Channel Attention Networks**. Proceedings of the European Conference on Computer Vision (ECCV), 2018. Disponível em: <https://arxiv.org/abs/1807.02758>. Acesso em: 14 mai. 2025.

ZHAO, X. et al. **A review of convolutional neural networks in computer vision**. Artificial Intelligence Review, v. 57, art. 99, 2024. Disponível em: <https://link.springer.com/article/10.1007/s10462-024-10721-6>. Acesso em: 02 mai. 2025.

ZHOU, C; GU, Z; GAO, Y; WANG, J. **An Improved Style Transfer Algorithm Using Feedforward Neural Network for Real-Time Image Conversion**. Sustainability, 2019. Disponível em <https://doi.org/10.3390/su11205673>. Acesso em 1 ago. 2025

REFERÊNCIAS (cronológica)

LECUN, Y. et al. **Gradient-Based Learning Applied to Document Recognition**. Proceedings of the IEEE, v. 86, n. 11, p. 2278–2324, 1998. Disponível em: <https://ieeexplore.ieee.org/document/726791>. Acesso em: 02 mai. 2025.

FARNEBÄCK, G; BIGUN, J; GUSTAVSSON, T. **Two-Frame Motion Estimation Based on Polynomial Expansion**. Image Analysis, 2003. Disponível em: https://doi.org/10.1007/3-540-45103-X_50. Acesso em: 06 abr. 2025

WANG, Z. et al. **Image Quality Assessment: From Error Visibility to Structural Similarity**. IEEE Transactions on Image Processing, v. 13, n. 4, p. 600–612, 2004. Disponível em: <https://www.cns.nyu.edu/pub/eero/wang03-reprint.pdf>. Acesso em: 02 mai. 2025.

HAYKIN, S. **Neural Networks and Learning Machines**. 3ª ed., Pearson, 2009. Disponível em: <http://dai.fmph.uniba.sk/courses/NN/haykin.neural-networks.3ed.2009.pdf>. Acesso em: 02 mai. 2025.

FLYNN, J.R. et al. **Image Quality Assessment Using the SSIM and the Just Noticeable Difference Paradigm**, 2013. Disponível em: https://link.springer.com/content/pdf/10.1007/978-3-642-39360-0_3.pdf. Acesso em: 20 set. 2025.

FISCHER, P; ALEXEY, D et al. **FlowNet: Learning Optical Flow with Convolutional Networks**. ArXiv, 2015. Disponível em: <https://arxiv.org/abs/1504.06852>. Acesso em: 06 abr. 2025.

MACKIN, A; ZHANG ,F; BULL, D. R. **A study of subjective video quality at various frame rates**. IEEE INTERNATIONAL CONFERENCE ON IMAGE PROCESSING (ICIP), 2015, Quebec City. Disponível em: <https://scispace.com/pdf/a-study-of-subjective-video-quality-at-various-frame-rates-hx11d80oi0.pdf>. Acesso em: 02 mai. 2025.

ZHOU, C.; GU, Z.; GAO, Y.; WANG, J. **An Improved Style Transfer Algorithm Using Feedforward Neural Network for Real-Time Image Conversion**. Sustainability, 2019. Disponível em <https://doi.org/10.3390/su11205673>. Acesso em 1 ago. 2025

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep Learning**. MIT Press, 2016. Disponível em: <https://www.deeplearningbook.org/>. Acesso em: 02 mai. 2025.

HE, K. et al. **Deep Residual Learning for Image Recognition**. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016. Disponível em: <https://arxiv.org/abs/1512.03385>. Acesso em: 14 mai. 2025.

SHI, W.; CABALLERO, J.; HUSZÁR, F.; TOTZ, J.; AITKEN, A.; BISHOP, R.; RUECKERT, D.; WANG, Z. **Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network**. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016. Disponível em: <https://arxiv.org/abs/1609.05158>. Acesso em: 14 mai. 2025.

JOHNSON, J. et al. **Perceptual Losses for Real-Time Style Transfer and Super-Resolution**. ECCV, 2016. Disponível em: <https://cs.stanford.edu/people/jcjohns/papers/eccv16/JohnsonECCV16.pdf>. Acesso em: 02 mai. 2025.

BARRON, J. T. **A General and Adaptive Robust Loss Function**. arXiv preprint arXiv:1701.03077, 2017. Disponível em: <https://arxiv.org/abs/1701.03077>. Acesso em: 03 mai. 2025.

DAI, J. et al. **Deformable Convolutional Networks**. ICCV, 2017. Disponível em: <https://arxiv.org/abs/1703.06211>. Acesso em: 02 mai. 2025.

LEDIG, C.; THEIS, L.; HUSZÁR, F.; CABALLERO, J.; CUNNINGHAM, A.; ACOSTA, A.; AITKEN, A.; TEJANI, A.; TOTZ, J.; WANG, Z.; SHI, W. **Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network**. arXiv preprint arXiv:1609.04802, 2017. Disponível em: <https://arxiv.org/pdf/1609.04802v5>. Acesso em: 02 mai. 2025.

SIMON, N. et al. 2017. **Video frame interpolation via adaptive convolution**. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 670–679. Disponível em: <https://arxiv.org/abs/1703.07514>. Acesso em: 02 mai. 2025.

DEQUING, S; XIAONDONG, Y; MING-YU, L; JAN, K. **PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume**. ArXiv, 2018. Disponível em: <https://arxiv.org/abs/1709.02371>. Acesso em: 06 abr. 2025.

HU, J.; SHEN, L.; SUN, G. **Squeeze-and-Excitation Networks**. CVPR, 2018. Disponível em: https://openaccess.thecvf.com/content_cvpr_2018/papers/Hu_Squeeze-and-Excitation_Networks_CVPR_2018_paper.pdf. Acesso em: 02 mai. 2025.

HUAIZU, J. et al. 2018. **Super slomo: High quality estimation of multiple intermediate frames for video interpolation**. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 9000–9008. Disponível em: <https://arxiv.org/abs/1712.00080>. Acesso em: 02 mai. 2025.

WOO, S. et al. **CBAM: Convolutional Block Attention Module**. ECCV, 2018. Disponível em: https://openaccess.thecvf.com/content_ECCV_2018/papers/Sanghyun_Woo_Convolutional_Block_Attention_ECCV_2018_paper.pdf. Acesso em: 02 mai. 2025.

ZHANG, Y. et al. **Image Super-Resolution Using Very Deep Residual Channel Attention Networks**. Proceedings of the European Conference on Computer Vision (ECCV), 2018. Disponível em: <https://arxiv.org/abs/1807.02758>. Acesso em: 14 mai. 2025.

WENBO, B. et al. 2019. **Depth-aware video frame interpolation**. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 3703–3712. Disponível em: <https://arxiv.org/abs/1904.00830>. Acesso em: 02 mai. 2025.

CHOI, M.; KIM, H.; HAN, B.; XU, N.; LEE, K. M. **Channel Attention Is All You Need for Video Frame Interpolation**. Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence (AAAI-20), Vancouver, Canada, 2020. Disponível em: <https://cdn.aaai.org/ojs/6693/6693-13-9922-1-10-20200521.pdf>. Acesso em: 02 mai. 2025.

HYEONGMIN, L. et al. 2020. **AdaCoF: Adaptive collaboration of flows for video frame interpolation**. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 5316–5325. Disponível em: <https://arxiv.org/abs/1907.10244>. Acesso em: 02 mai. 2025.

MOSTAFA. **Depth map artefacts reduction: a review**. IET Image Process, 2020. Disponível em: https://www.researchgate.net/figure/Result-of-35-compared-to-90-and-49-for-filling-hole-pixels-and-filtering-the_fig2_341404281. Acesso em: 02 mai. 2025.

XIANHANG, C.; ZHENZHONG, C. 2020. **Video frame interpolation via deformable separable convolution**. In Proceedings of the AAAI Conference on Artificial Intelligence, v. 34, p. 10607–10614. Disponível em: <https://ojs.aaai.org/index.php/AAAI/article/view/6634>. Acesso em: 23 mai. 2025.

ZACHARY T; JIA D. **RAFT: Recurrent All-Pairs Field Transforms for Optical Flow**. ArXiv, 2020. Disponível em: <https://arxiv.org/abs/2003.12039>. Acesso em: 06 abr. 2025.

KIM, J.; LEE, C. **Reliable Perceptual Loss Computation for GAN-Based Super-Resolution With Edge Texture Metric**. ResearchGate, ago. 2021. Disponível em: https://www.researchgate.net/figure/Some-feature-maps-extracted-from-a-pre-trained-VGG-19-feature-extractor-of-the-DIV2K-0312_fig1_354174248. Acesso em: 05 maio 2025.

BRIEDIS, K. M. et al. **Neural Frame Interpolation for Rendered Content**. ACM Trans. Graph., Vol. 40, No. 6, Article 239, 2021. Disponível em: <https://studios.disneyresearch.com/app/uploads/2021/10/Neural-Frame-Interpolation-for-Rendered-Content.pdf>. Acesso em: 06 abr. 2025.

SPARACELLO, Vitale. **A gentle introduction to Geometric Deep Learning**. Medium, 22 mai. 2022. Disponível em: <https://medium.com/dataroots/a-gentle-introduction-to-geometric-deep-learning-by-vitale-sparacello-91e529cf439b>. Acesso em: 05 maio 2025.

HENRYRU. **The concepts of Deep Learning, Neural Networks, Linear regression.** Medium, 12 jun. 2022. Disponível em: https://medium.com/@cheerful_whip_wombat_910/the-concepts-of-deep-learning-472ff58488bb. Acesso em: 05 maio 2025.

CLOUARD, R. **Peak Signal-to-Noise Ratio (PSNR).** Pandore Documentation, 2023 Disponível em: <https://clouard.users.greyc.fr/Pandore/programmes/en/operatorsP0/ppsnr.html#:~:text=Because%20many%20signals%20have%20a,between%2030%20and%2040%20dB>. Acesso em: 16 mai. 2025.

DEGUZMAN, D. **The power of video in news media: how increased audience engagement drives success.** The Associated Press, 2024. Disponível em: <https://www.ap.org/insights/the-power-of-video-in-news-media-how-increased-audience-engagement-drives-success/>. Acesso em: 06 abr. 2025.

GEEKSFORGEEEKS. **What is a Pixel?**, 2024. Disponível em: <https://www.geeksforgeeks.org/what-is-a-pixel/>. Acesso em: 02 mai. 2025.

ZHAO, X. et al. **A review of convolutional neural networks in computer vision. Artificial Intelligence Review**, v. 57, art. 99, 2024. Disponível em: <https://link.springer.com/article/10.1007/s10462-024-10721-6>. Acesso em: 02 mai. 2025.

FIVEABLE. **Peak Signal-to-Noise Ratio (PSNR). Biomedical Engineering II – Key Terms**, 2025 Disponível em: <https://fiveable.me/key-terms/biomedical-engineering-ii/peak-signal-to-noise-ratio-psnr>. Acesso em: 16 mai. 2025.

MILVUS. **What is a Patch in Image Processing?** AI Quick Reference, 2025 Disponível em: <https://milvus.io/ai-quick-reference/what-is-a-patch-in-image-processing>. Acesso em: 16 mai. 2025.